



*Asociación Profesional del Cuerpo Superior
de Sistemas y Tecnologías de la Información
de la Administración del Estado*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

Tema 66. Entorno de desarrollo Microsoft .NET

Editado por ASTIC en 2017 a partir de los temas numerados anteriormente como:

- **Tema 59. Entorno de desarrollo Microsoft .NET para software con distribución**
Autor: Sergio García Tobar
Actualización 2014
- **Tema 115. Arquitectura de desarrollo en la WEB (II). Tecnología .NET**
Autor: Sergio García Tobar
Actualización 2014

INDICE

1. Introducción
2. El framework .NET
3. ASP .NET
4. ADO .NET
5. Arquitectura de desarrollo en la WEB. Tecnología .NET
6. Arquitectura de aplicaciones distribuidas
7. La capa de presentación
8. La capa de negocio
9. La capa de servicios
10. La capa de datos
11. Arquitectura física
12. Bibliografía

1. Introducción

La revolución de la tecnología de la información, juntamente con las comunicaciones y el inseparable desarrollo del canal Internet, ha cambiado la forma en que se desarrollan y se utilizan las aplicaciones.

Con la aparición de la tecnología Internet, el modelo para los sistemas de información ha variado, desde el modelo cliente/servidor, el cual se caracterizaba por ser muy complejo, costoso y propietario, al paradigma actual centrado en el modelo Web. Los sistemas de información han evolucionado a modelos de 3 capas... *n*-capas poco acopladas (*loosely-coupled*). Una capa (*layer*) es un conjunto de componentes software agrupados por su funcionalidad, que no tiene por qué ejecutarse necesariamente en un único servidor (*tier*). Una aplicación Web típica consta de 3 capas: la capa de presentación, la capa de negocio y la capa de datos. Sin embargo, alguna de estas capas puede dividirse a su vez en otras capas, dando lugar a las arquitecturas de *n* capas.

La extensión de las aplicaciones distribuidas y la necesidad de integrar aplicaciones heterogéneas propició la aparición de los servicios Web y las tecnologías SOA (*Service-Oriented Architecture*). En la actualidad, existe un creciente interés por las aplicaciones en la nube y en general el *cloud computing*, en los tres modelos que adopta: SaaS (*software as a service*), PaaS (*platform as a service*) e IaaS (*infrastructure as a service*).

En este contexto se incardinan las tecnologías de Microsoft .NET, que se despliegan en los siguientes ámbitos:

- Aplicaciones Web: ASP.NET.
- Servicios: ASP.NET y WCF.
- Aplicaciones en la nube: Azure.
- Aplicaciones para dispositivos móviles: Silverlight, XNA.
- Aplicaciones de escritorio: WinForms, WPF.

Todas estas tecnologías se basan en el framework .NET.

2. El framework .NET

En los sistemas Windows, la alternativa a la programación de aplicaciones en .NET es utilizar directamente **WINAPI** (*Windows API*) en C++, que permite un acceso a bajo nivel del sistema, por ejemplo, en lo referente a la gestión de memoria y el acceso al hardware. Las librerías de .NET utilizan WINAPI; sin embargo, el framework .NET es más que una capa de abstracción sobre WINAPI, ya que presenta sistemas propios de un auténtico sistema operativo, como son los de gestión de memoria y de ejecución aislada de aplicaciones en dominios, su propio modelo de *threading* y otros que se describirán a continuación.

El framework .NET se compone de una biblioteca de clases, denominada **FCL** (*Framework Class Library*), y del entorno común de ejecución **CLR** (*Common Language Runtime*).

Con CLR se pueden utilizar diferentes lenguajes de programación, como C#, Visual Basic, F# o C++, que deben cumplir con las normas definidas en la **CLS** (*Common Language Specification*). Microsoft ha implementado CLR como un servidor COM dentro de una librería DLL (*mscorlib.dll* o *clr.dll* según la versión de CLR). El resto de componentes del framework están implementados también como librerías DLL que se ejecutan en modo usuario. En el mismo Windows es posible tener instalados distintos frameworks .NET, alguno de los cuales están integrados en el propio sistema operativo. Así, en Windows 8.1 y Windows Server 2012 R2 está presente el framework .NET 4.5.1 y en Windows 8 o Windows Server 2012 el 4.5, pero es posible instalar en todos ellos la versión del framework .NET 3.5.

En .NET el resultado de la compilación de las aplicaciones es un módulo administrado (realmente un ensamblado) en la forma de un fichero PE32 o PE32+ (*Windows portable executable*) que requiere del entorno CLR para ejecutarse. Es decir, mientras que un compilador nativo de C++ produce código para ejecutarse en una arquitectura específica de CPU (x86, x64, ARM, etc.), todos los compiladores del CLR producen en su lugar código IL, que se suele denominar **código administrado**. El compilador de C++ de Microsoft es el único que permite tener código administrado y no administrado en un mismo módulo. El código no administrado no necesita el entorno CLR para ejecutarse. Pero ambos códigos se pueden integrar porque un código no administrado puede hospedar el CLR llamando a la función *CLRCreateInstance* declarada en el fichero *MetaHost.h* (contenido en el Windows SDK).

Además de generar código IL, los compiladores del CLR generan metadatos dentro de cada módulo administrado. El compilador **JIT** (*just-in-time*) compila en tiempo de ejecución el código IL a instrucciones nativas de la CPU. Se requieren tantas compilaciones JIT como instancias de la aplicación se estén ejecutando. El **GC** (*Garbage Collector*) de CLR se encarga de la liberación de memoria, a cuyos efectos clasifica los objetos en tres generaciones. Las recolecciones de basura se desencadenan por las siguientes causas: cuando se intenta reservar memoria para un nuevo objeto y se sobrepasa el límite de memoria designado a la generación en cuestión (256K para la generación 0, 2 MB para la generación 1 y 10 MB para la generación 2); cuando se invoca *GC.Collect()* en el código; y cuando el sistema operativo genera una alerta de falta de memoria.

La unidad de ejecución de CLR es el **ensamblado**, que se puede definir como el agrupamiento de módulos administrados y ficheros de recursos. Cada ensamblado tiene un fichero de manifiesto que contiene todos los metadatos necesarios para especificar los requisitos de versión y la identidad de seguridad del ensamblado, y todos los metadatos necesarios para definir el ámbito del ensamblado y resolver las referencias a los recursos y las clases. Los ensamblados se utilizan para el empaquetamiento físico y la implementación y puede contener tipos, el código ejecutable utilizado para implementarlos y referencias a otros ensamblados. Existen dos clases principales de ensamblados: aplicaciones y bibliotecas. Las aplicaciones tienen un punto de entrada principal y normalmente tienen la extensión de archivo *.exe*; las bibliotecas no tienen un punto de entrada principal y suelen tener la extensión de archivo *.dll*.

Entre las ventajas más destacadas de utilizar código IL en vez de código nativo se pueden señalar las siguientes. En primer lugar, IL es un código portable: si la aplicación se está ejecutando, por ejemplo, en una versión x86 de Windows, JIT generará instrucciones x86. En segundo lugar, se asegura la estabilidad de la aplicación a través del proceso de **verificación**

que se realiza cada vez que se compila código IL en código nativo. La verificación se basa en los metadatos de los módulos administrados. Por ejemplo, la verificación comprueba que cada parámetro pasado a un método es del tipo correcto. Y en tercer lugar, el compilador JIT comprueba también en la verificación la seguridad del código antes de ejecutarlo, permitiendo sólo la ejecución del código inseguro expresamente autorizado. CLR admite un modelo de seguridad denominado seguridad de acceso del código o **CAS** (*Code Access Security*) para el código administrado. En este modelo se conceden permisos a los ensamblados basados en la identidad del código. La directiva de seguridad que determina los permisos que se conceden a los ensamblados se define en tres sitios distintos: la directiva de equipo, la directiva de usuario para el código administrado que se hospeda en un proceso y la directiva de host de CLR, que está activa para el código administrado que se ejecuta en ese host (por ejemplo, SQL Server). Por defecto, cada ensamblado .exe se ejecuta en su propio **dominio de aplicación** (*AppDomain*) –en su propio proceso de Windows–. Sin embargo, los procesos host de CLR, como IIS (*Internet Information Services*) o Microsoft SQL Server, pueden decidir ejecutar varios dominios de aplicación en el mismo proceso de Windows.

Estos conceptos de .NET operan en el siguiente ejemplo de código escrito en C#. CLR compilará el código en el ensamblado *thread3.exe*. En su ejecución, el compilador JIT cargará dos ensamblados, cada uno en su propio dominio de aplicación (*thread3.exe* y *mscorlib.dll*, que corresponde al espacio de nombres *System.Threading* que forma parte de la FCL de .Net) y compilará los dos métodos (*Main()* y *ThreadProc()*). Por otra parte, el método *start* de la clase *Thread* internamente llama a la función *CreateThread* de WINAPI. Además, la mayoría de los métodos de la clase *Thread* están protegidos por CAS, de tal manera que el código hospedado dentro de un programa como SQL Server no puede iniciar o controlar un *thread*.

```
using System;
using System.Threading;
public class thread3 {
    public static void ThreadProc() {
        for (inti = 0; i < 10; i++) {
            Console.WriteLine("ThreadProc: {0}", i);
            Thread.Sleep(0);
        }
    }

    public static void Main() {
        Console.WriteLine("Main thread: Start a second thread.");
        Thread t = new Thread(new ThreadStart(ThreadProc));
        t.Start();
        for (inti = 0; i < 4; i++) {
            Console.WriteLine("Main thread: Do some work.");
            Thread.Sleep(0);
        }

        Console.WriteLine("Main thread: Call Join(), to wait until ThreadProc ends.");
        t.Join();
        Console.WriteLine("Main thread: ThreadProc.Join has returned. PressEnter to endprogram.");
        Console.ReadLine();
    }
}
```

El resultado de la ejecución de este código es el siguiente:

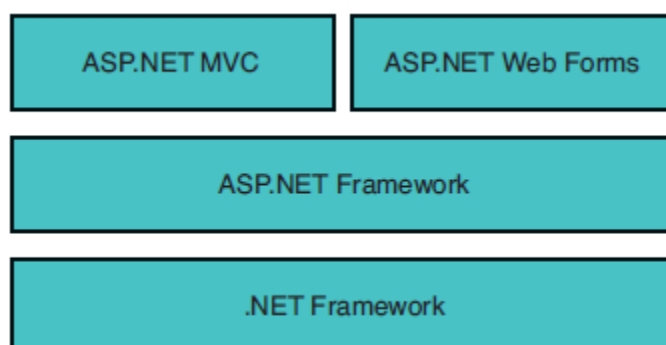
```
Main thread: Start a second thread.  
Main thread: Do some work.  
Main thread: Do some work.  
Main thread: Do some work.  
Main thread: Do some work.  
Main thread: Call Join(), to wait until ThreadProc ends.  
ThreadProc: 0  
ThreadProc: 1  
ThreadProc: 2  
ThreadProc: 3  
Main thread: ThreadProc.Join has returned. Press Enter to end program.
```

3. ASP .NET

ASP.NET es un conjunto de tecnologías del framework .NET para construir aplicaciones Web y servicios Web. Ambos se despliegan en el servidor de aplicaciones **IIS** (*Internet Information Services*). El ensamblado *System.Web.dll* contiene todas las clases de ASP.NET.

Una **aplicación Web ASP.NET** es la “suma de todos los archivos, páginas, manejadores de eventos, módulos y código ejecutable que pueden ser invocados en el entorno de un directorio virtual de un servidor web”. Se compone principalmente de **páginas .aspx**, de **ensamblados .dll** y del fichero **Web.config**. Los ensamblados son el resultado de compilar las clases con la lógica de presentación de las páginas .aspx, las clases de negocio y las clases de acceso a datos. *Web.config* es un fichero xml que contiene configuración de la aplicación, de seguridad y de acceso a los proveedores de datos, y se puede encontrar en diferentes subcarpetas de la aplicación a las que extiende sus efectos. IIS utiliza asimismo los ficheros de configuración a nivel de host (*aspnet.config*) y de equipo (*machine.config*).

Existen dos frameworks para el desarrollo de aplicaciones Web en ASP.NET: ASP.NET Web Forms y ASP.NET MVC.



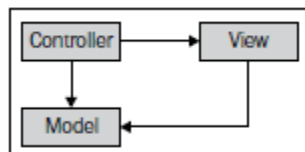
Los **formularios Web ASP.NET** constan de dos partes: las páginas.aspx y las clases asociadas a las mismas. Las primeras contienen la vista de la interfaz de usuario, que puede consistir en: código HTML, directivas, *scriptlets* y controles Web. Una directiva importante es *Page*: `<%@ Page %>`. Las clases asociadas a las páginas .aspx son subclases de *System.Web.UI.Page*, que contienen los métodos para responder a los eventos del ciclo de vida de la página y de los controles Web. El atributo *Inherits* de la directiva *Page* permite indicar qué clase debe heredar la nueva clase que se va a generar dinámicamente a partir de una página .aspx. Los *scriptlets* son código .NET en elementos `<script runat="server">` o `<%>`.

Los **controles Web** son clases que se ejecutan en el servidor y devuelven código HTML al navegador web del cliente. Se clasifican en diferentes grupos: controles estándar (como las etiquetas, los botones o los cuadros de textos), controles de validación (como el validador de rango), controles enriquecidos (como el calendario o el botón de carga de ficheros), controles de datos (como la rejilla de datos), controles de navegación (como el menú o la vista en árbol), controles de login (como la vista de login) y controles HTML. Los controles de usuario son controles Web definidos por los desarrolladores (.ascx).

El framework **ASP.NET Web Forms** se basa en los controles Web, en el mantenimiento del estado entre peticiones y en la programación dirigida por eventos. Para el mantenimiento del estado, ASP.NET proporciona diversas técnicas, como las variables de estado de vista (*ViewState*), la inserción de datos en URL, las variables de sesión, las variables de aplicación y las cookies. Durante el proceso de petición de un formulario Web se producen una serie de eventos en el servidor que están definidos en la clase *Page*. Alguno de los eventos más significativos son el *Init*, que se produce al recibir la petición de la página .aspx en el servidor durante el proceso de inicialización de la misma; el evento *Load*, al cargar en memoria la página y el evento *PreRender*, que se produce cuando se va a proceder al envío de la respuesta al cliente. Para responder a estos eventos se definen unos métodos en la subclase *Page* del archivo de código asociado.

ASP.NET MVC implementa el patrón de diseño MVC (Modelo-Vista-Controlador). MVC consiste en tres tipos de objetos. El Modelo contiene la información que maneja la aplicación, incluyendo la de negocio y la lógica de acceso a la misma; la Vista es su representación en pantalla y el Controlador define el modo en que la interfaz reacciona a la entrada del usuario. MVC desacopla las vistas de los modelos estableciendo entre ellos un protocolo de suscripción/notificación y encapsula el mecanismo de respuesta a la entrada de usuario en un objeto Controlador. El modelo se puede implementar de manera activa o pasiva. El modelo activo notifica a la vista cuando es modificado por el controlador, mientras que en la implementación pasiva del modelo es el controlador quien notifica a la vista cuando modifica el modelo.

La relación entre estos tres componentes se muestra en la siguiente figura:



Se puede observar que el modelo no depende de la vista ni del controlador, y que la vista es independiente del controlador y se actualiza cuando cambia el modelo.

En ASP.NET MVC, los **controladores** son subclases de la clase *System.Web.Mvc.Controller* que implementan los métodos que responden a las acciones de usuario o a las llamadas de las vistas. El controlador y la acción por defecto se definen en el fichero *Global.asax.cs*. El controlador puede pasarle datos obtenidos del modelo a la vista mediante el objeto *ViewBag* o utilizando una clase del Modelo. Las **vistas** pueden ser páginas *.aspx* o ficheros *.cshtml* que utilizan el motor de vistas *Razor*. El **modelo** está formado por clases que mapean las tablas de la base de datos mediante una herramienta ORM (*Object-Relational Mapping*) como *Microsoft LINQ to SQL*, *NHibernate* o *ADO.NET Entity Framework*. Las cadenas de conexión se definen dentro del fichero *Web.config*.

El framework MVC supone, aparte de la separación entre la interfaz de usuario y la lógica de presentación y el modelo -las vistas son páginas *.aspx* sin la lógica de presentación, que reside en el controlador, como se muestra en el siguiente figura-, el cambio de un diseño basado en páginas a uno basado en la arquitectura REST (*Representational State Transfer*), de tal manera que el mapeo de las rutas se realiza según el esquema {controlador}/{acción}/{id}. Por ejemplo, en la figura siguiente la URL de la vista por defecto es <http://localhost/asp4/>, en vez de <http://localhost/asp4/Index.aspx>.



Respecto a los ámbitos de utilización de los dos frameworks comentados, el framework ASP.NET Web Forms es más adecuado en desarrollos RAD (*Rapid Application Development*), mientras que el framework ASP.NET MVC facilita las pruebas automáticas de software.

Por último, nos referiremos a los **servicios Web ASP.NET**.

Los servicios Web XML permiten la integración de sistemas heterogéneos mediante el intercambio de mensajes SOAP sobre HTTP. En ASP.NET, son ficheros *.asmx* que incluyen la directiva *WebService* y que, al igual que las páginas *.aspx*, pueden tener clases asociadas. A la

clase que implementa el servicio Web se le añade el atributo *WebService* y al método expuesto el atributo *WebMethod*. La enumeración *WsiProfiles* describe la especificación de interoperabilidad de los servicios Web (WSI) a la que un servicio Web solicita la conformidad.

El siguiente ejemplo corresponde a un servicio Web que expone el método *GetRandom* que genera una cadena aleatoria a partir de un número entero que se le pasa como parámetro. El servicio Web está implementado por los ficheros *Consulta.asmx* y *Consulta.cs*.

El código de **Consulta.asmx** es:

```
<%@ WebService Language="C#" CodeBehind="Consulta.cs" Class="Principal.Consulta" %>
```

Y el código de **Consulta.cs**:

```
using System;
using System.Web;
using System.Collections;
using System.Web.Services;
using System.Web.Services.Protocols;
using System.Text;
using System.Drawing;
namespace Principal
{
    [WebService(Namespace = "http://consulta.principal/")]
    [WebServiceBinding(ConformsTo = WsiProfiles.BasicProfile1_1)]
    public class Consulta : System.Web.Services.WebService
    {
        [WebMethod]
        public string GetRandom(int number)
        {
            Generador mgenerador = new Generador();
            return mgenerador.GetRandomString(number);
        }
    }
}
```

El cliente SOAP puede ser una clase proxy en C# o en cualquier otro lenguaje. En este caso, se utilizará el módulo SOAP::Lite de Perl:

```
use SOAP::Lite +trace => [qw(debug)];
my $soap = SOAP::Lite
->proxy('http://localhost/Ticker2/consulta.asmx?WSDL')
->uri('http://consulta.principal/')
->on_action( sub { join '/', 'http://consulta.principal', $_[1] });
$soap->default_ns('http://consulta.principal/');
$soap->outputxml('true');
my @params = (SOAP::Data->name("number" => "7"));
my $result = $soap->call("GetRandom"=> @params);
for($result){
    s/^s*//;
    s/s*$//;
};
```

Los mensajes SOAP intercambiados se muestran en esta figura:

```
SOAP::Transport::HTTP::Client::send_receive: POST http://localhost/Ticker2/consulta.asmx?WSDL
HTTP/1.1
Accept: text/xml
Accept: multipart/*
Accept: application/soap
Content-Length: 462
Content-Type: text/xml; charset=utf-8
SOAPAction: http://consulta.principal/GetRandom

<?xml version="1.0" encoding="UTF-8"?><soap:Envelope xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:soapenc="http://schemas.xmlsoap.org/soap/encoding/" xmlns:xsd="http://www.w3.org/2001/XMLSchema" soap:encodingStyle="http://schemas.xmlsoap.org/soap/encoding/" xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/"><soap:Body><GetRandom xmlns="http://consulta.principal/"><number xsi:type="xsd:int">7</number></GetRandom></soap:Body></soap:Envelope>
SOAP::Transport::HTTP::Client::send_receive: HTTP/1.1 200 OK
Cache-Control: private, max-age=0
Date: Tue, 23 Sep 2014 14:54:24 GMT
Server: Microsoft-IIS/7.5
Content-Length: 481
Content-Type: text/xml; charset=utf-8
Client-Date: Tue, 23 Sep 2014 14:54:24 GMT
Client-Peer: 127.0.0.1:80
Client-Response-Num: 1
X-AspNet-Version: 4.0.30319
X-Powered-By: ASP.NET

<?xml version="1.0" encoding="utf-8"?><soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance" xmlns:xsd="http://www.w3.org/2001/XMLSchema"><soap:Body><GetRandomResponse xmlns="http://consulta.principal/"><GetRandomResult>000Y 137,31 +6,81% ZSTM 173,41 +9,51% RPU 167,63 -9,41% ZKS 356,15 -0,75% UXM 314,02 -1,58% FNU0 80,3 -5,48% ICZ 273,34 -7,25%</GetRandomResult></GetRandomResponse></soap:Body></soap:Envelope>
```

ASP.NET se puede integrar con tecnologías como AJAX o Silverlight para construir aplicaciones RIA (*rich Internet application*), que mejoran la experiencia de usuario al aproximarla a la de una aplicación de escritorio.

El ensamblado de **ASP.NET AJAX** (*System.Web.Extensions*) generará automáticamente un proxy Javascript para los servicios que se registren utilizando la etiqueta `<asp:ServiceReference>` y utilizará el control `Web UpdatePanel` para llamar periódicamente al servicio y refrescar la interfaz de usuario. Considerando el ejemplo anterior, el código AJAX que habría que añadir en una página `.aspx` sería el siguiente:

```
<asp:ScriptManager runat="server" ID="scriptManager">
<Services>
<asp:ServiceReference Path="consulta.asmx"/>
</Services>
</asp:ScriptManager>

<asp:UpdatePanel ID="PanelConsulta" runat="server">
<ContentTemplate>
<asp:Timer ID="RefreshTimer" runat="server" Interval="3000"
OnTick="RefreshTimer_Tick"></asp:Timer>
<asp:TextBox ID="PanelConsultaQueue" runat="server" Width="500">
</asp:TextBox>
</ContentTemplate>
</asp:UpdatePanel>
```

ASP.NET también se puede integrar con librerías Javascript como **jQuery** o **Dojo**, que tienen sus propios frameworks AJAX.

La limitación que presentan los servicios Web al utilizar solamente el protocolo HTTP es superada en .NET por los **servicios WCF** (*Windows Communication Foundation*), que pueden ejecutarse sobre TCP, HTTP, IPC o MSMQ. WCF es un framework para construir aplicaciones orientadas a servicios. Mediante WCF se pueden enviar datos como mensajes asíncronos entre los endpoints de dos servicios, que pueden estar hospedados en IIS o en otro servidor.

Los **servicios Web WCF** pueden ser SOAP o REST; en este último caso, los mensajes intercambiados pueden tener el formato XML sin estar ensobrados en SOAP, el formato XML ATOM o un formato no XML como JSON.

El siguiente código corresponde a la implementación WCF REST del servicio Web del ejemplo anterior.

- **RestServiceImpl.svc:**

```
<%@ ServiceHost Language="C#" Debug="true" Service="RestService.RestServiceImpl"
CodeBehind="RestServiceImpl.svc.cs" %>
```

-**RestServiceImpl.svc.cs:**

```
using System;
namespace RestService
{
    public class RestServiceImpl : IRestServiceImpl
    {
        #region IRestServiceImpl Members

        public string xmlGetRandom(string number)
        {
            Generador mgenerador = new Generador();
            return mgenerador.GetRandom(Convert.ToInt32(number));
        }

        public string jsonGetRandom(string number)
        {
            Generador mgenerador = new Generador();
            return mgenerador.GetRandom(Convert.ToInt32(number));
        }

        #endregion
    }
}
```

- **IRestServiceImpl.cs:**

```
using System.ServiceModel;
using System.ServiceModel.Web;

namespace RestService
{
    [ServiceContract]
    public interface IRestServiceImpl
    {
        [OperationContract]
        [WebInvoke(Method = "GET",
            ResponseFormat = WebMessageFormat.Xml,
            BodyStyle = WebMessageBodyStyle.Wrapped,
            UriTemplate = "xml/{number}")]
        string xmlGetRandom(string number);

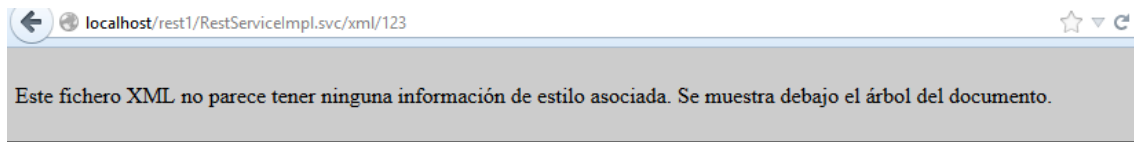
        [OperationContract]
        [WebInvoke(Method = "GET",
            ResponseFormat = WebMessageFormat.Json,
            BodyStyle = WebMessageBodyStyle.Wrapped,
            UriTemplate = "json/{number}")]
    }
}
```

```
stringjsonGetRandom(string number);  
}  
}
```

- **Web.config:**

```
<?xml version="1.0"?>  
<configuration>  
  <system.web>  
    <compilation debug="true" targetFramework="4.0" />  
  </system.web>  
  <system.serviceModel>  
    <services>  
      <service name="RestService.RestServiceImpl" behaviorConfiguration="ServiceBehaviour">  
        <!-- Service Endpoints -->  
        <endpoint address="" binding="webHttpBinding" contract="RestService.IRestServiceImpl"  
          behaviorConfiguration="web">  
        </endpoint>  
      </service>  
    </services>  
    <behaviors>  
      <serviceBehaviors>  
        <behavior name="ServiceBehaviour">  
          <serviceMetadatahttpGetEnabled="true"/>  
          <serviceDebugincludeExceptionDetailInFaults="false"/>  
        </behavior>  
      </serviceBehaviors>  
      <endpointBehaviors>  
        <behavior name="web">  
          <webHttp/>  
        </behavior>  
      </endpointBehaviors>  
    </behaviors>  
    <serviceHostingEnvironmentmultipleSiteBindingsEnabled="true" />  
  </system.serviceModel>  
  <system.webServer>  
    <modules runAllManagedModulesForAllRequests="true"/>  
  </system.webServer>  
</configuration>
```

En la siguiente imagen se muestra el resultado de consumir este servicio Web mediante REST XML.



```

- <xmlGetRandomResponse>
- <xmlGetRandomResult>
  RNO 328,66 -8,21% CFL 31,56 -1,66% LJH 345,64 -6,27% YFB 192,19 +2,24% EIL 127,49 -9,54% ZTJG 103,86 +6,12
  -5,99% QCWM 229,73 +4,24% LAVQ 57,25 +2,2% TXS 103,31 -3,55% XRLI 356,57 +8,83% CYGW 395,78 -6,98% JI
  +8,87% FRB 297,67 -1,9% YCX 160,03 +2,72% ALC 136,01 +3,73% ONXB 303,66 +6,69% GHPW 372,88 -1% WKD
  361,86 -2,43% MXGE 199,49 -6% JTS 17,47 +5,47% KQY 90,07 -8,3% SHO 158,35 +6,36% DACZ 190,08 +6,01% NF
  BKYT 43,77 -2,75% BRFR 368,44 +6,27% FBS 25,02 -8,82% QKI 346,87 +8,33% SCI 301,82 +3,77% RVH 276,59 -0,
  -5,29% KJA 333,54 +8,56% HZCJ 150,48 -5,98% LNJZ 120,03 -0,53% ODLR 55,87 +8,28% KGE 26,63 -2,66% BMM
  76,48 -3,02% VPUI 88,7 +8,23% LEH 135,48 -0,07% SWVR 225,23 -7,51% CUK 154,95 -9,77% NJP 234,05 -5,89% B
  +7,54% PKDV 229,28 +8,9% LTG 101,1 +2,54% DUD 343,07 +3,4% BXM 167,76 +3,13% EQYT 371,1 +7,92% RVXV
  170,33 -3,27% ZITG 174,59 -3,64% FVWD 337,46 +5,51% LMI 262,08 -2,73% JQDV 88,79 -6,01% FEDX 17,14 -0,88
  +7,08% NVJR 54,5 +2,94% BYQR 81,95 +8,81% RMNJ 75,66 -0,92% MWQ 108,93 +3,66% BTRU 218,48 +2,52% UN
  EIWK 333,43 +8,78% RRM 339,92 +4,45% QVPQ 167,02 +5,95% VBJP 184,17 +9,77% LTP 14,35 -2,07% ISM 146,35
  177,64 +2,79% YOCS 157,07 +8,79% ZBDR 212,82 -9,99% WDT 347,85 +0,51% VRKG 129,37 -3,09% DHZ 211,63 -
  374,62 +3,83% TZZA 195,76 -2,88% QVG 377,82 +4,13% KZV 67,17 +7% BFUR 75,25 -8,48% VOJY 36,12 -9,65% U
  </xmlGetRandomResult>
</xmlGetRandomResponse>

```

Y a continuación se muestra el resultado de consumir este servicio Web mediante REST JSON.

```

{"jsonGetRandomResult": "RKJP 8,52 +1,1% HPV 271,24 +4,12% MWD 96,38 -6,44% QFS 213,87 +2,24% ONZS 335,
MYXG 286,37 -8,27% TERL 157,25 +5,44% SBEN 273,44 +7,07% DRZ 252,93 -6,6% YXPZ 295,55 -6,8% JWC 61,61
RUKA 327,34 +6,33% MRGX 81,76 -8,14% EZJG 166,02 -4,88% OFGR 214,3 -4,59% SSF 5,02 -4,48% UAPJ 234,49
105,26 +7,95% SIJF 119,66 +1,84% ALIR 278,09 +9,84% GBO 376,25 -3,44% IDP 5,02 -4,22% AOSK 302,37 -2,3
171,66 -4,15% ZHXW 379,08 +3,97% SAM 121,59 -9,59% ZKK 220,99 +3,08% XJWE 107,46 +4,23% YRI 168,65 -8,
205,46 -8,12% STQW 199,49 -2,06% JSKN 350,71 -1,71% POXB 288,3 +1,28% PXH 270,26 -0,27% KBAF 145,49 +9
39,79 -4,32% AUCQ 175,58 -6,41% XTL 222,49 +7,35% OSR 80,97 -7,55% TKR 213,88 +8,85% SHZP 100,16 +6,92
189,07 +5,3% MWB 71,46 -2,01% QXQZ 287,87 -5,31% CWEM 53,29 +6,42% TPH 143,18 -6,93% HTH 52,02 +1,83%
148,24 -4,35% RTN 247,54 +5,64% BNZT 166,6 +1,33% QTZH 355,02 +0,27% HRP 29,25 +9,97% RUTH 225,07 -5,6
+6,67% OLBS 128,74 -1,06% RLMS 205,87 -2,97% UHWF 23,47 +9,98% BPK 53,28 -4,37% OMLD 87,36 -0,89% OLL
-0,04% FCL 277,75 -3,07% PZAV 236,07 -2,95% SINL 11,74 +4,96% NKJP 187,09 +7,03% JLM 361,23 -9,17% VVS
-6,13% NUX 277,25 +3,26% VIPI 290,8 +2,76% UTRM 113,59 -9,58% DZCK 61,1 -5,4% YAG 272,8 -0,41% BQGX 39
+6,85% RSCP 333,34 -2,75% QFVJ 157,35 -8,89% QHR 17,62 -3,37% YNT 196,71 -2,11% OKB 168,98 -7,72% EVH
+6,65% HZW 46,88 -4,66% JCO 67,95 +0,69% FXX 328,56 +4,55% LYY 137,72 +1,02% UDVR 344,51 -8,97% SLBB 2

```

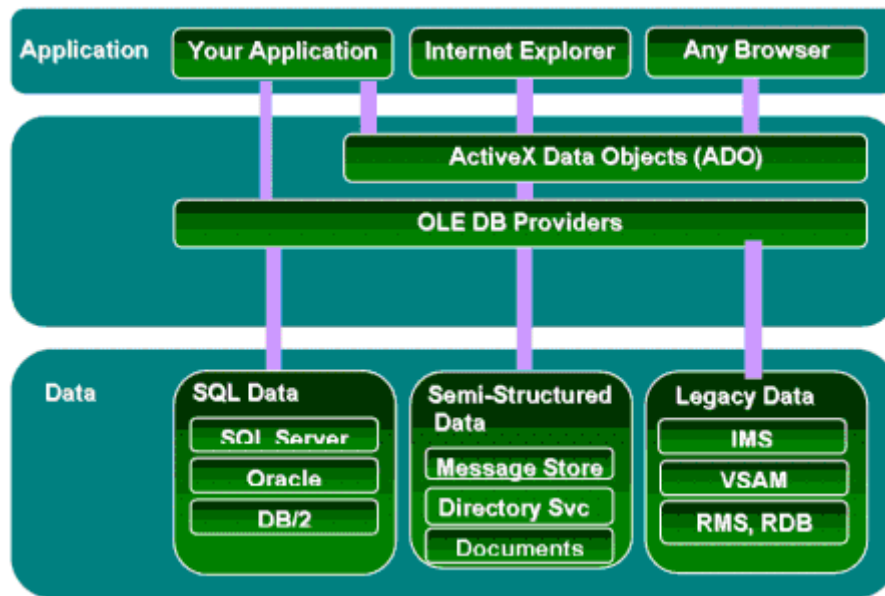
Aunque Microsoft recomienda utilizar para desarrollar aplicaciones distribuidas, WCF, también se puede utilizar **.NET Remoting** (*System.Runtime.Remoting*), que consiste en un conjunto de servicios y canales de comunicación –como el canal TCP– que transmiten mensajes entre aplicaciones remotas, con la ayuda de formateadores que codifican y decodifican esos mensajes.

4. ADO .NET

A continuación se presenta una breve descripción de la evolución de las tecnologías de Microsoft de acceso a las Bases de Datos para pasar a explicar en más detalle ADO.NET:

- **ODBC (Open Database Connectivity)**
 - Es una interfaz común de acceso a múltiples Bases de Datos (BBDD) usando SQL.
 - Es una interfaz C++ de bajo nivel.
 - Usa un **driver específico** para cada BBDD concreta. Permite conectar con cualquier base de datos de la que exista un driver ODBC. Los creadores de las distintas bases de datos son los responsables de crear un driver ODBC para que su base de datos se pueda conectar desde un sistema Microsoft.
 - Permite cambiar de BBDD sin recompilar ni modificar la aplicación.
- **DAO (Data Access Object) y RAO (Remote Data Object)**
 - Es una colección de objetos de más alto nivel que ODBC para usar con lenguajes VB, Delphi, etc.
 - Está basado en el Motor Jet (Access).
 - Puede atacar otras Bases de Datos, pero mediante conversión Jet a ODBC. Sin embargo, esto implica más capas y más lento.
 - RDO creado para no usar Jet y usar directamente ODBC.
- **OleDB**
 - Es un interfaz de **acceso universal a datos** (BBDD, ficheros, Exchange, Directorios, etc).
 - Es un interfaz para lenguajes tipo C++.
- **ADO (Active Data Object)**
 - Es una arquitectura de objetos para simplificar el uso de OleDB.
 - Utilizable en lenguajes tipo VB y de script.
 - Por debajo puede usar OleDB u ODBC.
 - Se basa en **RecordSets**, conectados formados por una sola tabla. Estos RecordSets son poco escalables.

En el siguiente gráfico vemos como las aplicaciones se conectan a los datos haciendo uso de ADO y OleDB:



➤ RDS (Remote Data Sources)

- Intento de mejorar ADO.
- Permite RecordSets desconectados.

➤ ADO.NET

ADO.NET proporciona **acceso coherente a orígenes de datos como Microsoft SQL Server y XML**, así como a orígenes de datos expuestos mediante OLE DB y ODBC. Las aplicaciones para usuarios que comparten datos pueden utilizar ADO.NET para conectar a estos orígenes de datos y recuperar, manipular y actualizar los datos contenidos.

ADO.NET separa el acceso a datos de la manipulación de datos y crea componentes discretos que se pueden utilizar por separado o conjuntamente. ADO.NET incluye proveedores de datos de .NET Framework para conectarse a una base de datos, ejecutar comandos y recuperar resultados. Los resultados se procesan directamente o se colocan en un objeto **DataSet** de ADO.NET con el fin de exponerlos al usuario para un propósito específico, combinados con datos de varios orígenes, o de utilizarlos de forma remota entre niveles. El objeto **DataSet** de ADO.NET también puede utilizarse independientemente de un proveedor de datos de .NET Framework para administrar datos que son locales de la aplicación o que proceden de un origen XML.

▪ Componentes de ADO.NET

Existen dos componentes de ADO.NET que se pueden utilizar para obtener acceso a datos y manipularlos:

- 1.- Proveedores de datos de .NET Framework
- 2.- El **DataSet**

1.- Proveedores de datos de .NET Framework

Los **proveedores de datos de .NET Framework** son componentes diseñados explícitamente para la manipulación de datos y el acceso rápido a datos de sólo lectura y sólo avance. El objeto **Connection** proporciona conectividad a un origen de datos. El objeto **Command** permite tener acceso a comandos de base de datos para devolver datos, modificar datos, ejecutar procedimientos almacenados y enviar o recuperar información sobre parámetros. El objeto **DataReader** proporciona una secuencia de datos de alto rendimiento desde el origen de datos. Por último, el objeto **DataAdapter**

proporciona el puente entre el objeto **DataSet** y el origen de datos. El **DataAdapter** utiliza objetos **Command** para ejecutar comandos SQL en el origen de datos tanto para cargar el **DataSet** con datos como para reconciliar en el origen de datos los cambios aplicados a los datos incluidos en el **DataSet**.

Los proveedores de datos en el framework .NET 4.5 son los siguientes:

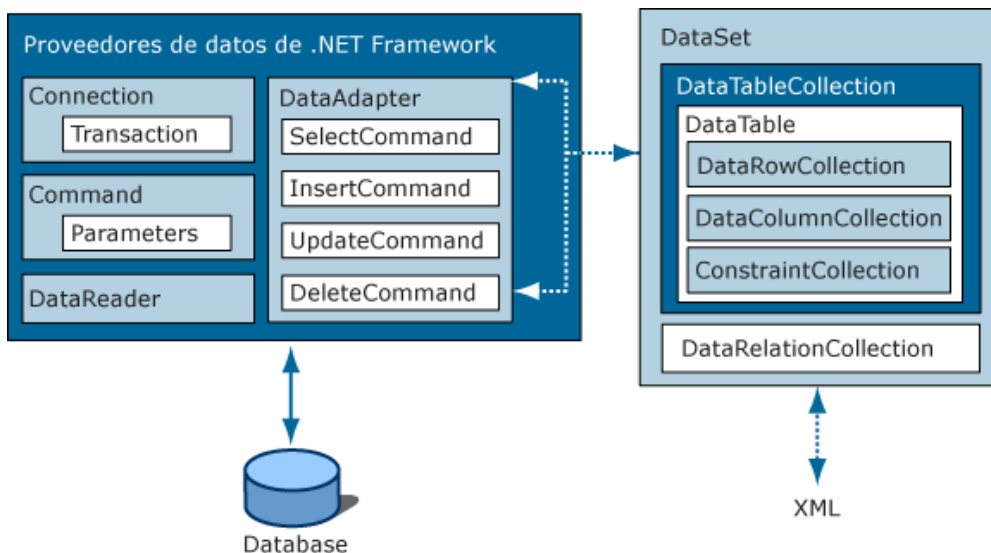
- *Proveedor de datos para SQL Server*: proporciona acceso a datos para Microsoft SQL Server. Utiliza el espacio de nombres *System.Data.SqlClient*.
- *Proveedor de datos para OLE DB*: para orígenes de datos que se exponen mediante OLE DB. Utiliza el espacio de nombres *System.Data.OleDb*.
- *Proveedor de datos para ODBC*: para orígenes de datos que se exponen mediante ODBC. Utiliza el espacio de nombres *System.Data.Odbc*.
- *Proveedor de datos para Oracle*: Para orígenes de datos de Oracle. Utiliza el espacio de nombres *System.Data.OracleClient*.
- *Proveedor EntityClient*: proporciona acceso a datos para las aplicaciones de Entity Data Model (EDM). Utiliza el espacio de nombres *System.Data.EntityClient*.
- *Proveedor de datos para SQL Server Compact 4.0*: proporciona acceso de datos para Microsoft SQL Server Compact 4.0. Usa el espacio de nombres *System.Data.SqlServerCe*.

2.- DataSet

El **DataSet** de **ADO.NET** está expresamente diseñado para el acceso a datos independientemente del origen de datos. Como resultado, se puede utilizar con múltiples y distintos orígenes de datos, con datos XML o para administrar datos locales de la aplicación. El **DataSet** contiene una colección de uno o más objetos **DataTable** formados por filas y columnas de datos, así como información sobre claves principales, claves externas, restricciones y relaciones relativas a los datos incluidos en los objetos **DataTable**.

En el diagrama siguiente se ilustra la relación entre un proveedor de datos de .NET Framework y un **DataSet**.

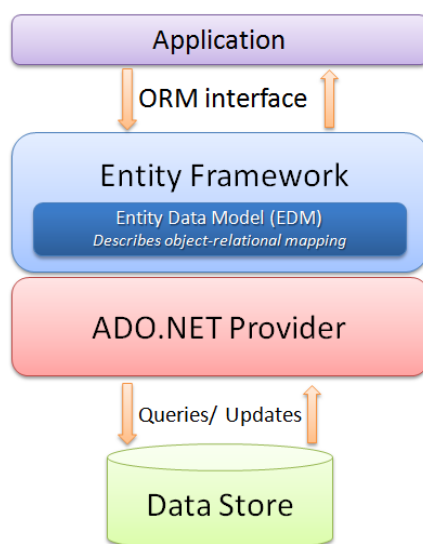
En la siguiente figura se muestra la arquitectura de ADO.NET:



En el siguiente ejemplo se utiliza **ADOMD.NET**, que es un proveedor de datos .NET diseñado para comunicarse con Microsoft SQL Server Analysis Services.

```
AdomdCommandcmd = new AdomdCommand();  
cmd.CommandText = "select [Customer].MEMBERS on ROWS, [Product].MEMBERS on  
COLUMNS from [Sales] where [Measures].[Store Cost]";  
AdomdDataAdapter da = new AdomdDataAdapter(cmd);  
DataSet ds = new DataSet();  
da.Fill(ds,"Medida");  
System.Windows.Forms.DataGrid vista;  
vista.DataSource = ds;
```

A partir de la versión 3.5, el framework .NET incluye el componente **ADO.NET Entity**, que es framework ORM (*Object/Relational Mapping*) que permite el desarrollo de aplicaciones de software orientadas a datos, reduciendo la cantidad de código y el mantenimiento necesarios. Por ejemplo, la inserción de datos en una base de datos se realiza mediante *AddObject(Objectentity)*.

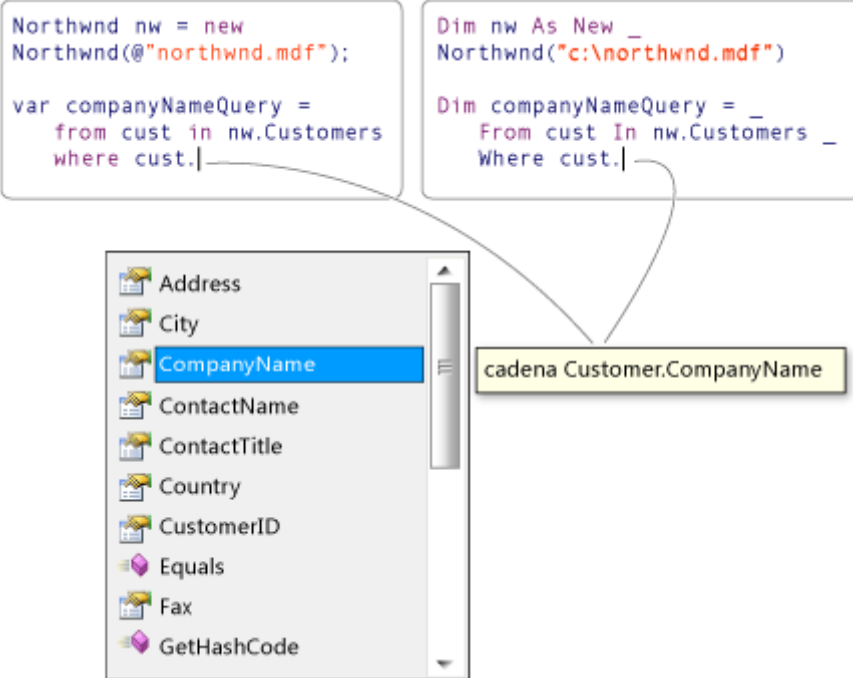


Sobre esta pila de frameworks, formada por ADO.NET y Entity, se sitúa el framework **WCF Data Services** (anteriormente conocido como *ADO.NET Data Services*), que es también un componente del framework .NET. Permite crear servicios que utilizan *Open Data Protocol* (OData) para exponer y utilizar datos a través de web o de una intranet utilizando REST (*Representational State Transfer*). OData expone los datos como recursos direccionables a través de URIs (*Uniform Resource Identifier*). Para tener acceso a los datos y cambiarlos se utilizan los verbos HTTP estándar GET, PUT, POST y DELETE. OData usa las convenciones de entidad-relación de *Entity Data Model* para exponer los recursos como conjuntos de entidades que están relacionadas por medio de asociaciones.

La versión del framework .NET 3.5 también introdujo la innovación del framework **LINQ** (*Language-Integrated Query*). Mediante LINQ se pueden ejecutar consultas a diferentes orígenes de datos desde los lenguajes de programación .NET, como en la siguiente figura, que muestra una consulta LINQ parcialmente completada en una base de datos SQL Server en C#. Existen implementaciones de LINQ para diferentes data sources:

- LINQ to Objects
- LINQ to ADO.NET
- LINQ to Entities
- LINQ to SQL
- LINQ to DataSet

- LINQ to XML



5. Arquitectura de desarrollo en la WEB. Tecnología .NET

El diseño de la arquitectura de un sistema es el proceso por el cual se define una solución para los requisitos técnicos y operacionales del mismo. Este proceso define qué componentes forman el sistema, cómo se relacionan entre ellos, y cómo mediante su interacción llevan a cabo la funcionalidad especificada, cumpliendo con criterios de calidad, seguridad, disponibilidad, eficiencia o usabilidad. Los **patrones de diseño** pueden definirse como colaboraciones entre clases e instancias, y se pueden explicar mediante los diagramas de colaboración. Los patrones o **estilos de arquitectura** (*architectural style*) se utilizan de forma parecida, pero entre subsistemas o incluso sistemas. Un estilo de arquitectura es un conjunto de principios sobre cómo dividir el sistema en subsistemas y cómo estos subsistemas deben interactuar, a fin de conseguir las ventajas que el estilo proporciona. Los estilos de arquitectura se pueden clasificar según su área principal de interés, como se muestra en la tabla siguiente:

Category	Architecture styles
Communication	Service-Oriented Architecture (SOA), Message Bus
Deployment	Client/Server, N-Tier, 3-Tier
Domain	Domain Driven Design
Structure	Component-Based, Object-Oriented, Layered Architecture

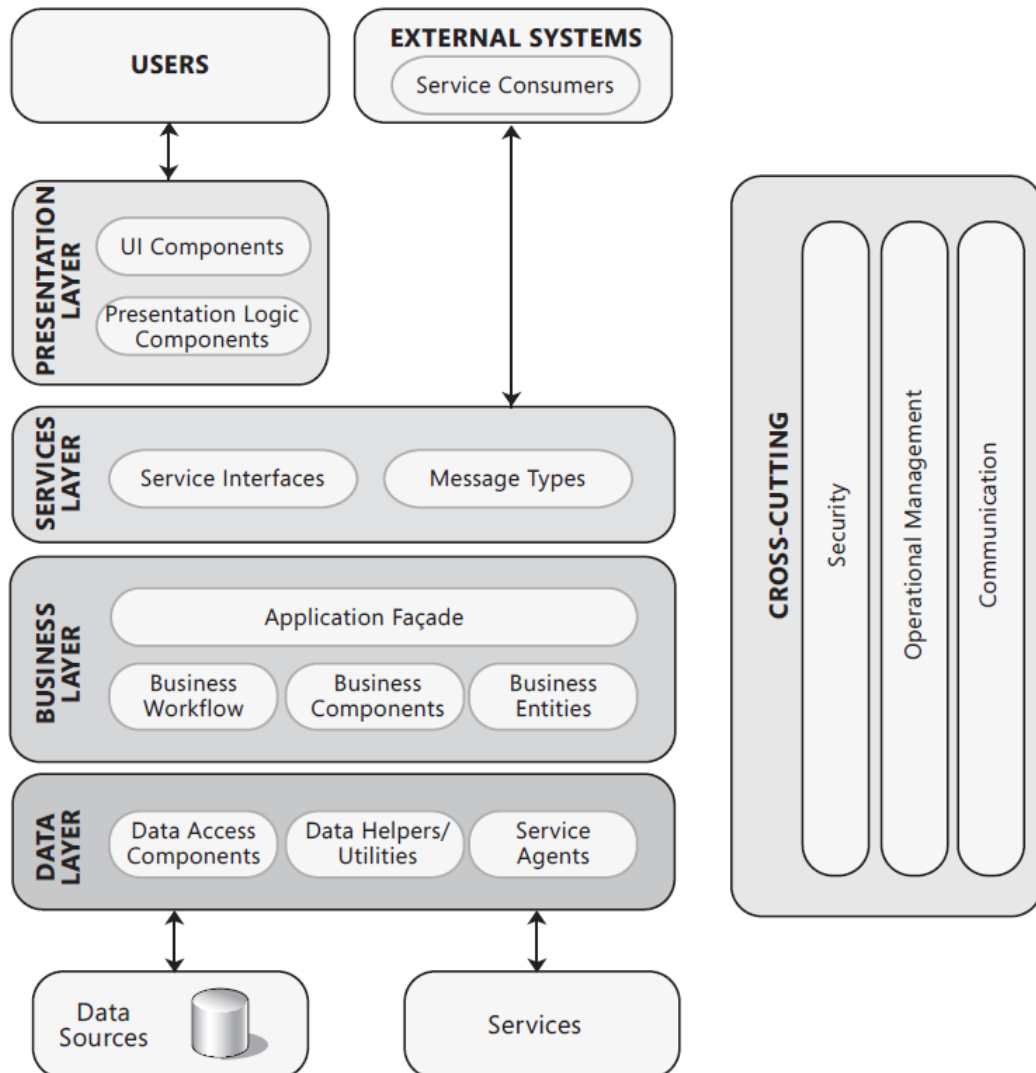
En los siguientes párrafos se utilizará principalmente el **patrón de arquitectura en capas** (*Layers*), que es aplicable a muchos tipos de sistemas y que es, además, el más referido en el libro que se mencionará a continuación. Este patrón define cómo organizar el modelo de diseño en capas, lo cual quiere decir que los componentes de una capa sólo pueden hacer referencia a componentes en capas inmediatamente inferiores. La comunicación entre capas está basada en una abstracción que proporciona un bajo acoplamiento entre capas, de forma que las capas más bajas no son conscientes de ningún detalle o interfaz de las superiores. Además, nos permite identificar qué partes de un sistema pueden reutilizarse, y ayuda a tomar decisiones sobre qué partes comprar y qué partes construir.

En el año 2002, Microsoft publicó el libro "*Application Architecture for .NET: Designing Applications and Services*", que contenía una serie de reglas para desarrollar aplicaciones distribuidas utilizando el framework .NET. Posteriormente, en el año 2009, Microsoft procedió a revisar y actualizar esta arquitectura en su libro "*Microsoft Application Architecture Guide, 2nd Edition*", donde además de explicarse a nivel general los principios de diseño y patrones para el entorno .NET, se concretaron los mismos para los **arquetipos de aplicaciones** más comunes, como las aplicaciones Web, las aplicaciones enriquecidas, las aplicaciones para dispositivos móviles, los proveedores de servicios o los servicios hospedados y en la nube. Aunque estas reglas están escritas para el entorno .NET, la mayoría de ellas se pueden aplicar por analogía en otras plataformas de desarrollo de aplicaciones. En el desarrollo de este tema nos referiremos a esta arquitectura.

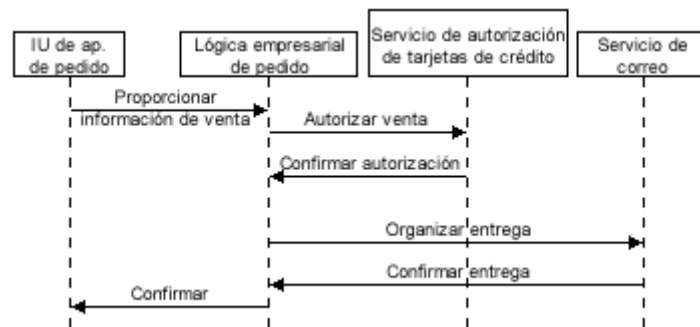
6. Arquitectura de aplicaciones distribuidas

En la siguiente figura se pueden ver los componentes software más comunes en una aplicación distribuida. Un **componente software** es un artefacto de implementación en el sentido utilizado en el RUP (*Rational Unified Process*), por lo que puede tratarse de un ensamblado, una página web o un producto de una plataforma empresarial. Los componentes software se agrupan en **capas** (*layers*) para ofrecer la funcionalidad del sistema acordada con el cliente, pero al mismo tiempo también se presta atención a la interacción entre los componentes y cómo éstos trabajan de manera solidaria. Estas capas son lógicas y pueden desplegarse en una o en varias **capas físicas** (*tiers*). Si una capa se puede ejecutar en un único proceso, se podrá

ejecutar por tanto en un único servidor (1-tier) o en varios servidores (n-tier), si se requiere el balanceo de carga o la alta disponibilidad de la capa. Sin embargo, aunque dos capas individualmente puedan ejecutarse en procesos separados, si su IPC (*inter-process communication*) se basa, por ejemplo, en memoria compartida local, tendrán que ejecutarse necesariamente en la misma tier.



Los componentes de la arquitectura se describirán desde la capa de presentación basándonos en el siguiente ejemplo. Consideremos una aplicación comercial de venta a través de la cual los clientes solicitan productos. La aplicación utiliza un servicio de autorización de tarjetas de crédito externas para validar los detalles de la tarjeta de crédito del cliente y autorizar la venta. Una vez comprobados los datos de la tarjeta de crédito, se utiliza un servicio de correo para organizar la entrega de los productos. El siguiente diagrama de secuencia muestra este escenario.



En la **capa de presentación** (*Presentation Layer*):

1. **Componentes de interfaz de usuario** (*UI Components*): La mayor parte de las soluciones necesitan ofrecer al usuario un modo de interactuar con la aplicación. En el ejemplo de la aplicación comercial, un sitio Web permite al cliente ver productos y realizar pedidos, y una aplicación basada en el entorno operativo Microsoft Windows permite a los representantes de ventas escribir los datos de los pedidos de los clientes que han telefonado a la empresa. Las interfaces de usuario permiten procesar y dar formato a los datos de los usuarios, así como adquirir y validar los datos entrantes procedentes de éstos.
2. **Componentes de lógica de presentación** (*Presentation Logic Components*): También se denominan componentes de proceso de usuario. En un gran número de casos, la interacción del usuario con el sistema se realiza de acuerdo con un proceso predecible. Por ejemplo, en la aplicación comercial, podríamos implementar un procedimiento que permita ver los datos del producto. De este modo, el usuario puede seleccionar una categoría de una lista de categorías de productos disponibles y, a continuación, elegir uno de los productos de la categoría seleccionada para ver los detalles correspondientes. Del mismo modo, cuando el usuario realiza una compra, la interacción sigue un proceso predecible de recolección de datos por parte del usuario, por el cual éste en primer lugar proporciona los detalles de los productos que desea adquirir, a continuación los detalles de pago y, por último, la información para el envío. Para facilitar la sincronización y organización de las interacciones con el usuario, resulta útil utilizar componentes de lógica de presentación individuales. De este modo, el flujo del proceso y la lógica de administración de estado no se incluye en el código de los elementos de la interfaz de usuario, por lo que varias interfaces podrán utilizar el mismo "motor" de interacción básica.

En la **capa de servicios** (*Services layer*):

3. **Interfaces de servicios** (*Service Interfaces*): Para exponer la lógica empresarial como un servicio, es necesario crear interfaces de servicios que admitan los contratos de comunicación (comunicación basada en mensajes, formatos, protocolos, seguridad y excepciones, entre otros) que requieren los clientes. Por ejemplo, el servicio de autorización de tarjetas de crédito debe exponer una interfaz de servicios que describa la funcionalidad que ofrece el servicio, así como la semántica de comunicación requerida para llamar al mismo. La capa de servicios ofrece un canal de acceso a la aplicación distinto de la capa de presentación. Está diseñada para el acceso desde otras aplicaciones o sistemas externos.
4. **Tipos de mensajes** (*MessageTypes*): Son los admitidos por los contratos de los servicios. Los tipos de mensajes intercambiados a través de las interfaces de servicio no deben revelar nunca detalles de los procesos de negocio o de las entidades empresariales utilizadas.

En la **capa de negocio** (*Business layer*):

5. **Fachada de la aplicación** (*Application Facade*): Es un componente opcional que se utiliza cuando la comunicación con los componentes de lógica de presentación requiere de la

composición de funcionalidad de varios componentes empresariales. Permite encapsular una operación de negocio que realmente consiste en un conjunto de operaciones entre varios componentes empresariales.

6. **Flujos de trabajo empresariales** (*Business Workflow*): Una vez que el proceso de usuario ha recopilado los datos necesarios, éstos se pueden utilizar para realizar un proceso empresarial. Por ejemplo, tras enviar los detalles del producto, el pago y el envío a la aplicación comercial, puede comenzar el proceso de cobro y preparación del envío. Gran parte de los procesos empresariales conllevan la realización de varios pasos, los cuales se deben organizar y llevar a cabo en un orden determinado. Por ejemplo, el sistema empresarial necesita calcular el valor total del pedido, validar la información de la tarjeta de crédito, procesar el pago de la misma y preparar el envío del producto. El tiempo que este proceso puede tardar en completarse es indeterminado, por lo que sería preciso administrar las tareas necesarias, así como los datos requeridos para llevarlas a cabo. Los flujos de trabajo empresariales definen y coordinan los procesos empresariales de varios pasos de ejecución larga y se pueden implementar utilizando herramientas de administración de procesos empresariales.
7. **Componentes empresariales** (*Business Components*): Independientemente de si el proceso empresarial consta de un único paso o de un flujo de trabajo organizado, la aplicación requerirá probablemente el uso de componentes que implementen reglas empresariales y realicen tareas empresariales. Por ejemplo, en la aplicación comercial, deberá implementar una funcionalidad que calcule el precio total del pedido y agregue el costo adicional correspondiente por el envío del mismo. Los componentes empresariales implementan la lógica empresarial de la aplicación.
8. **Entidades empresariales** (*Business Entities*): La mayoría de las aplicaciones requieren el paso de datos entre distintos componentes. Por ejemplo, en la aplicación comercial es necesario pasar una lista de productos de los componentes lógicos de acceso a datos a los componentes de la interfaz de usuario para que éste pueda visualizar dicha lista. Los datos se utilizan para representar entidades empresariales del mundo real, como productos o pedidos. Las entidades empresariales que se utilizan de forma interna en la aplicación suelen ser estructuras de datos, aunque también se pueden implementar utilizando clases orientadas a objetos personalizadas que representan entidades del mundo real necesarias para la aplicación, como productos o pedidos.

En la **capa de datos** (*data layer*):

9. **Componentes de acceso a datos** (*Data Access Components*). La mayoría de las aplicaciones y servicios necesitan obtener acceso a un almacén de datos en un momento determinado del proceso empresarial. Por ejemplo, la aplicación empresarial necesita recuperar los datos de los productos de una base de datos para mostrar al usuario los detalles de los mismos, así como insertar dicha información en la base de datos cuando un usuario realiza un pedido. Por tanto, es razonable abstraer la lógica necesaria para obtener acceso a los datos en una capa independiente de componentes lógicos de acceso a datos, ya que de este modo se centraliza la funcionalidad de acceso a datos y se facilita la configuración y el mantenimiento de la misma.
10. **Agentes de servicios** (*Service Agents*): Cuando un componente empresarial requiere el uso de la funcionalidad proporcionada por un servicio externo, tal vez sea necesario hacer uso de código para administrar la semántica de la comunicación con dicho servicio. Por ejemplo, los componentes empresariales de la aplicación comercial descrita anteriormente podrían utilizar un agente de servicios para administrar la comunicación con el servicio de autorización de tarjetas de crédito y utilizar un segundo agente de servicios para controlar las conversaciones con el servicio de mensajería. Los agentes de servicios permiten aislar las idiosincrasias de las llamadas a varios servicios desde la aplicación y pueden proporcionar servicios adicionales, como la asignación básica del formato de los datos que expone el servicio al formato que requiere la aplicación.

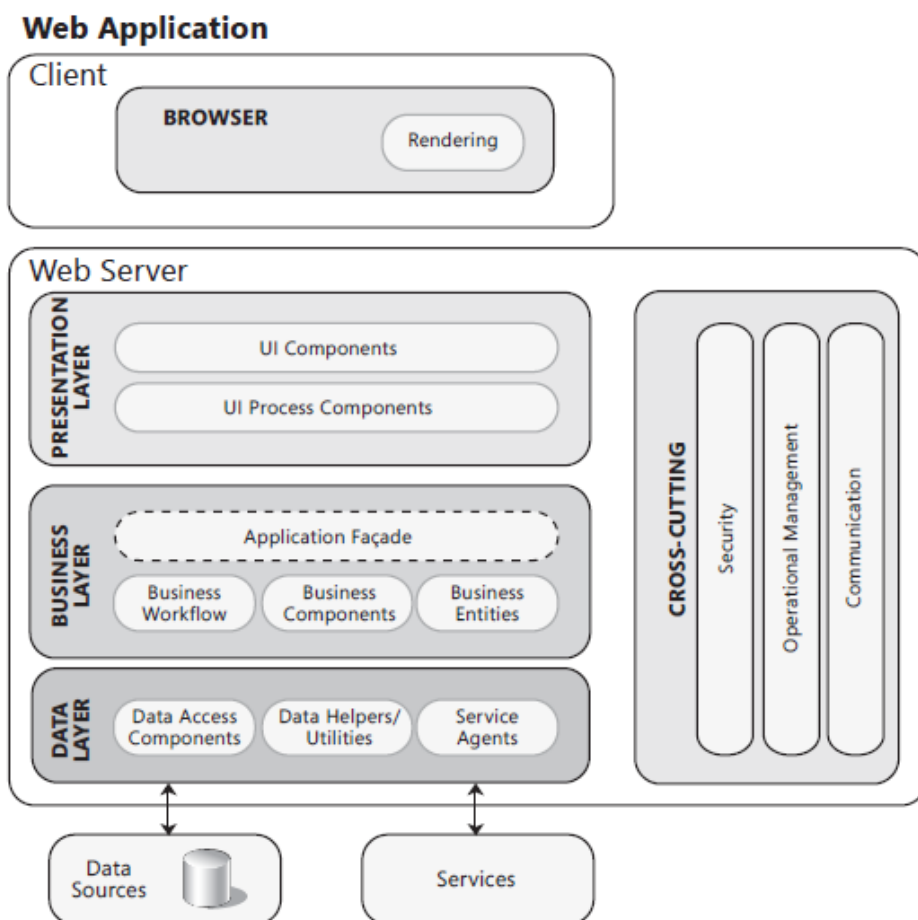
11. **Ayudas / Utilidades de acceso a datos** (*Data Helpers / Utilities*): como las definidas por algunos frameworks de acceso a datos, que encapsulan alguna funcionalidad del acceso a datos en estos componentes reutilizables.

En la **capa de infraestructura transversal** (*Cross Cutting*):

12. **Componentes de seguridad, administración operativa y comunicación** (*Security, Communication, Operational Management*). La aplicación probablemente utilice también componentes para realizar la administración de excepciones, autorizar a los usuarios a que realicen tareas determinadas y comunicarse con otros servicios y aplicaciones.

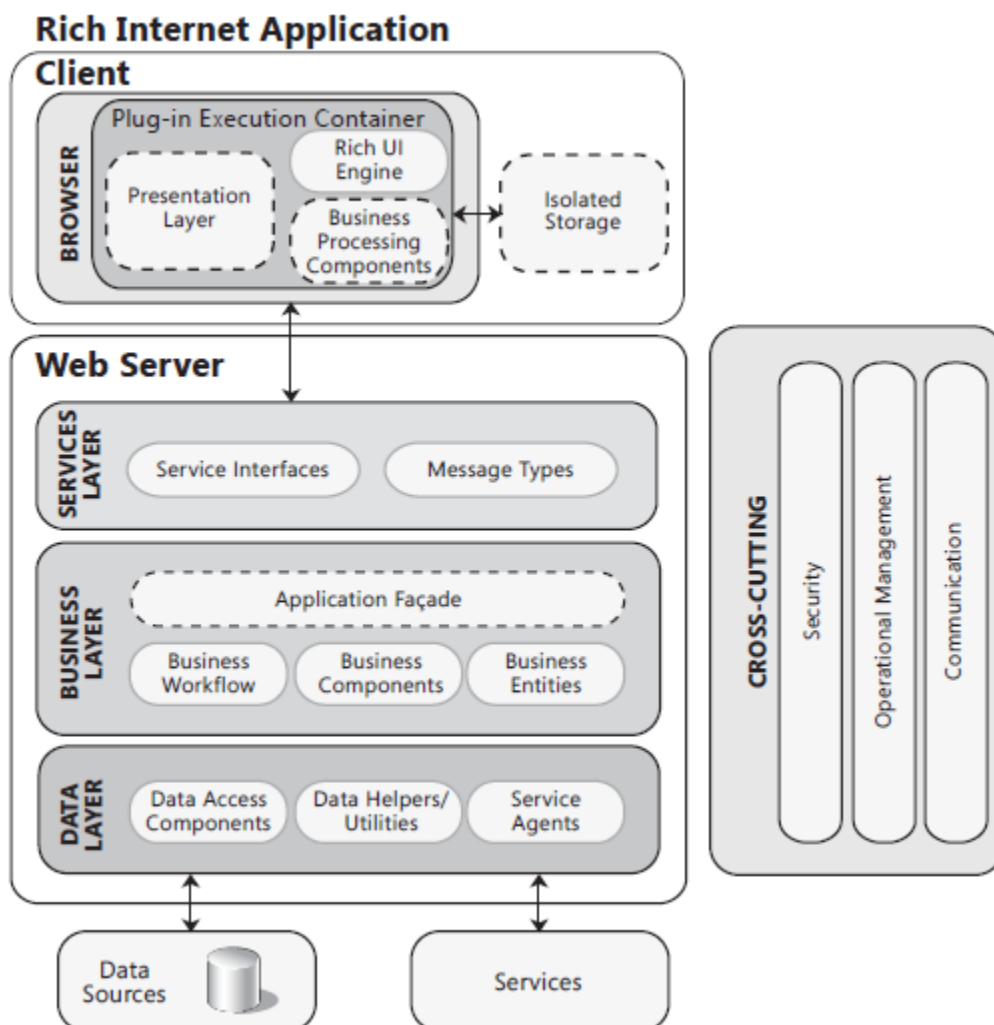
A continuación nos referiremos a la arquitectura de algunos arquetipos de aplicaciones.

Una **aplicación Web** es una aplicación que puede ser accedida por los usuarios mediante un navegador web o un agente de usuario especializado. El navegador web crea peticiones HTTP a URLs específicas que mapean los recursos en un servidor web/de aplicaciones. El servidor procesa estas peticiones y devuelve contenido HTML al navegador web. El núcleo de una aplicación Web es la lógica del lado de servidor. En el *cliente* residen los componentes del navegador web encargados de la presentación (*rendering*) del código HTML enviado por el servidor. El *servidor* de aplicaciones contiene la capa de presentación, la capa de negocio y la capa de datos. En la siguiente figura se muestra esta arquitectura. Los componentes con el contorno a trazos son opcionales.

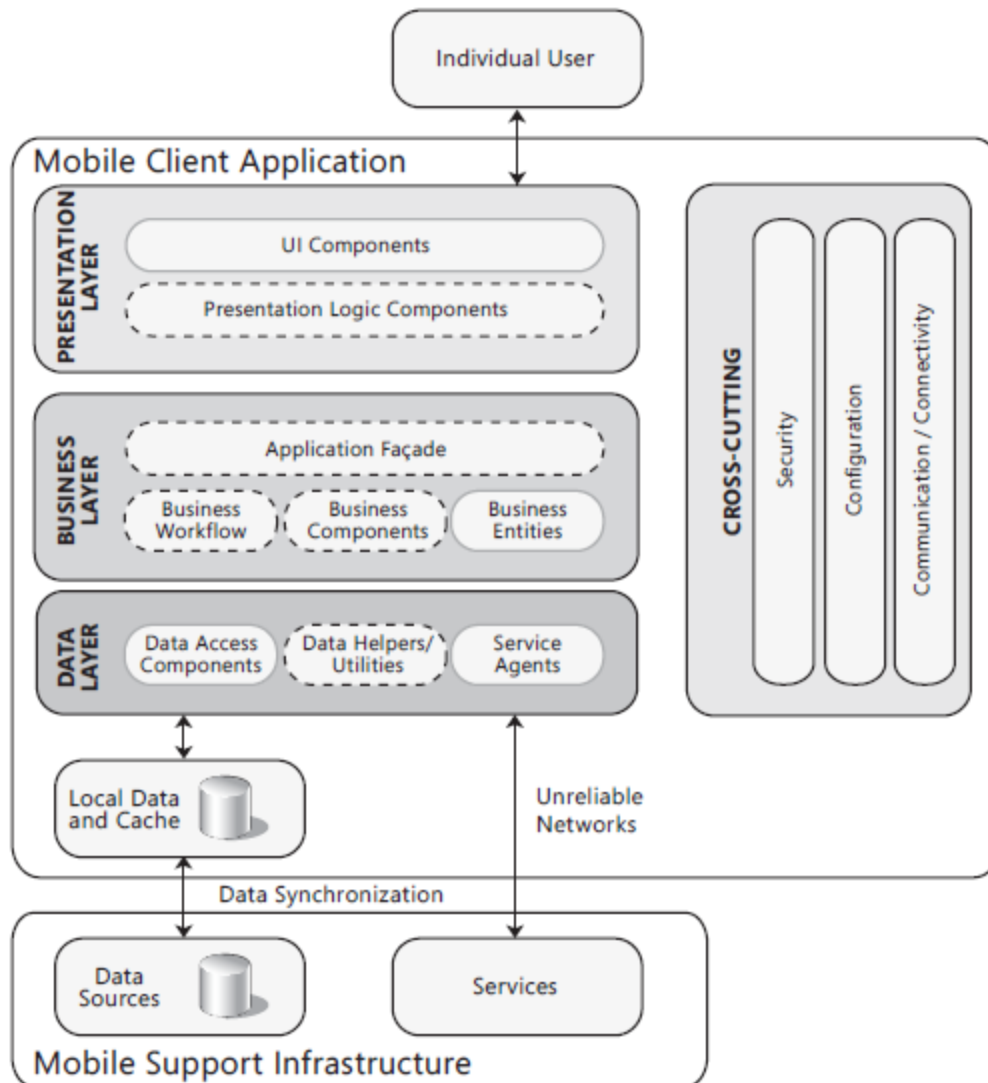


En el caso de una **aplicación enriquecida de Internet** (*Rich Internet Application*), en el *cliente* -en concreto, en el plugin del browser- reside la capa de presentación y la capa de negocio (componentes empresariales). Las aplicaciones RIA pueden contemplar escenarios de funcionamiento offline, por lo que se replican en el cliente ciertas partes de capas típicas del servidor, como la capa de datos, de tal manera que el almacenamiento aislado local está

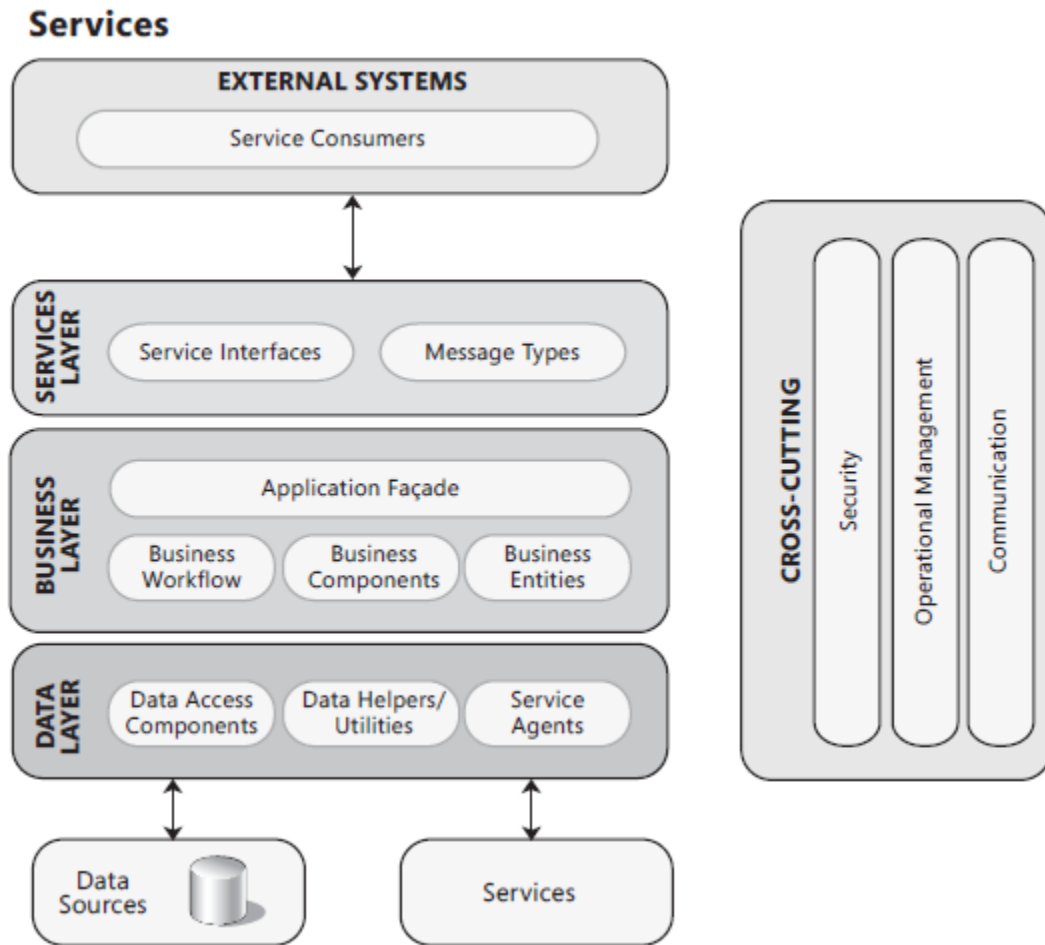
sincronizado con los *datasources* accesibles en el servidor. En el *servidor de aplicaciones* está implementada la capa de servicios, la capa de negocio (fachada de aplicación, flujos de trabajo empresariales, componentes empresariales y entidades empresariales) y la capa de datos. Esta arquitectura se muestra en la siguiente figura.



Las **aplicaciones para dispositivos móviles** (*mobile applications*) tienen en el cliente una capa de presentación constituida por componentes de interfaz de usuario y, opcionalmente, componentes de lógica de presentación; una capa de negocio que puede no existir y de hacerlo, normalmente comprende componentes empresariales, flujos de trabajo y entidades empresariales; y la capa de datos. En este tipo de aplicaciones se tiende a reducir el número de componentes y la separación en capas es menos rígida.



En una **aplicación que provee servicios** (*services application*), en el servidor hay tres capas: la capa de servicios, la capa de negocio y la capa de datos, como se muestra en la figura siguiente.

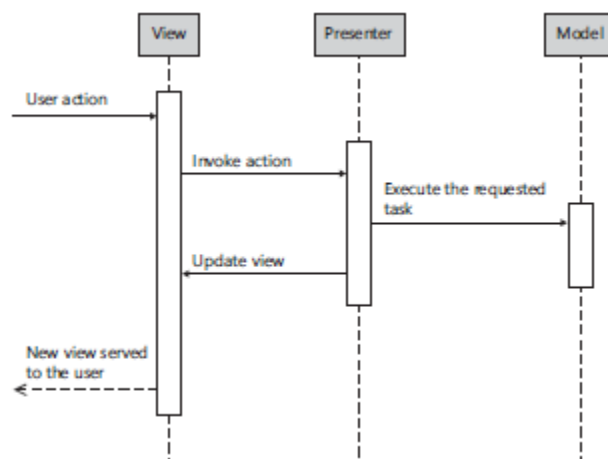


7. La capa de presentación

Los **UIC** (*User Interface Components*) desempeñan dos funciones: mostrar la información al usuario y capturar la interacción del usuario con la aplicación. Los **PLC** (*Presentation Logic Components*), contienen la lógica de presentación, que es distinta que la lógica de negocio, ya que su objetivo es sólo la presentación gráfica de la información al usuario, lo cual implica normalmente transformar la entrada de usuario en una entrada a las capas de servicios o de negocio. Esta subcapa PLC debe ser independiente de la UIC.

Los patrones de diseño de la capa de presentación más utilizados son: MVC (*Model-View-Controller*), MVP (*Model-View-Presenter*) y PM (*Presentation Model*).

El **MVC** se caracteriza porque extrae toda la lógica de presentación de las vistas. Los UIC son las vistas de página y los PLC las clases controlador. El **MVP** es una variante del MVC que solventa dos de las limitaciones de este modelo: permite las peticiones directas del controlador a las vistas y se posibilita también que, en algunos casos, los componentes UI –por ejemplo, *widgets*– respondan directamente a las acciones de usuario. Su funcionamiento se muestra en el siguiente diagrama de secuencia:



El **PM** es también una variación del MVC, especialmente indicado para interfaces enriquecidas como las construidas con WPF o Silverlight. En este caso, al contrario que en MVP, las vistas son pasivas y no exponen ninguna interfaz al presentador. En el presentador se incorpora un modelo de datos que representa el estado de las vistas, y las vistas se actualizan según este modelo, que ya no pertenece por tanto a la capa de negocio.

Entre las tecnologías .NET que se pueden utilizar para implementar la capa de presentación se pueden destacar las siguientes: **ASP.NET**, **WinForms**, **WPF** (*Windows Presentation Foundation*) o **Silverlight**. En ASP.NET se pueden utilizar formularios Web con controles Web como DataGrid y, en general, los definidos en el espacio de nombres *System.Web.UI*. Las interfaces de WPF y Silverlight están basadas en XAML (*Extensible Application Markup Language*).

8. La capa de negocio

Los **componentes empresariales** responden a las peticiones de los PLC. Pueden estar localizados en el cliente, en un servidor de aplicaciones o en ambos. En las aplicaciones RIA puede haber componentes empresariales en el cliente, pero si el componente tiene que acceder a un recurso no disponible desde el cliente, como el acceso a una base de datos, o por motivos de seguridad, estos componentes tendrán que desplegarse necesariamente en un servidor de aplicaciones. Las reglas de diseño de esta capa se refieren a la búsqueda del menor acoplamiento posible entre los componentes empresariales y los PLC, de la idempotencia o habilidad para ignorar peticiones duplicadas y de la garantía de servicio. Los componentes empresariales son responsables de las transacciones que ocurren en la capa de negocio, por lo que es necesario disponer de un **sistema de soporte a las transacciones**. Si una de las acciones a realizar en un proceso falla, tiene que existir la posibilidad de realizar un *rollback* de los efectos producidos por el resto de acciones del mismo proceso.

En .NET, el soporte de transacciones se ha implementado tradicionalmente mediante TSQL (en las propias sentencias y procedimientos almacenados SQL), transacciones ADO.NET (mediante los objetos *Connection* y *Transaction*) o transacciones COM+ con *Enterprise Services*. Sin embargo, a partir del framework .NET 2.0, existe el espacio de nombres **System.Transactions**, que aporta una gran flexibilidad a la hora de implementar transacciones.

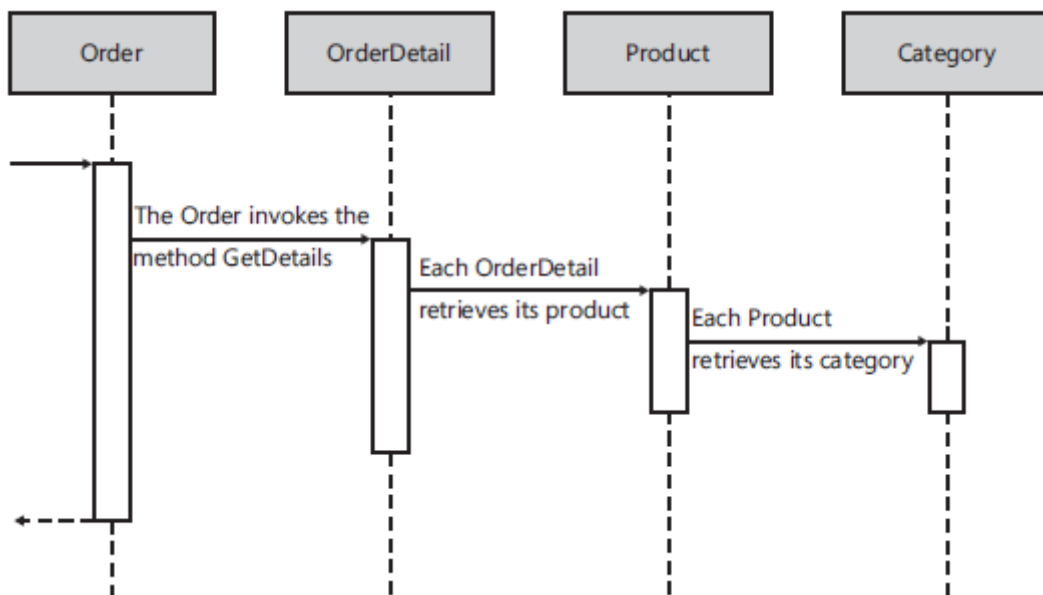
En el siguiente código, desde un componente empresarial se llama a dos servicios WCF en una única transacción. Para realizar el *commit* de la transacción se llama al método *Complete* del objeto *TransactionScope*, y para el *rollback* al método *Dispose*. Ambos servicios WCF tienen que estar configurados para soportar transacciones.

```

using (TransactionScopets = new TransactionScope(TransactionScopeOption.RequiresNew))
{
    try
    {
        ServiceReference1.Service1Client obj = new ServiceReference1.Service1Client();
        obj.UpdateData();
        ServiceReference2.Service1Client obj1 = new ServiceReference2.Service1Client();
        obj1.UpdateData();
        ts.Complete();
    }
    catch (Exception ex)
    {
        ts.Dispose();
    }
}

```

Las **entidades empresariales** almacenan y gestionan los datos que utilizan los procesos de negocio. En la siguiente figura se muestra un diagrama de secuencia entre varias entidades empresariales. La clase *Product* puede tener un atributo *Categoría* que de soporte a la asociación definida entre *Product* y *Category*, y un atributo *Descripción*, de tal manera que cuando se cree un producto haya que pasarle al constructor estos parámetros. Estas clases tienen métodos para escribir/leer los valores de sus atributos. El estilo de arquitectura **DDD** (*Domain Driven Design*) se centra en el modelo de dominio del sistema e identifica patrones de diseño como Entidad.



Un ejemplo de código de la clase *Order* es:

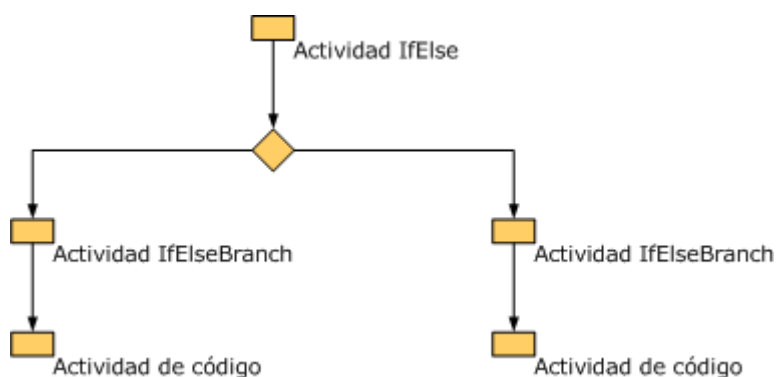
```

public partial class Order
{
    public int OrderId {get; set;}
    public int OrderStatusID {get; set;}
    public virtual OrderStatus OrderStatus {get; set;}
    public virtual ICollection<OrderDetail> GetDetails {get; set;}
}

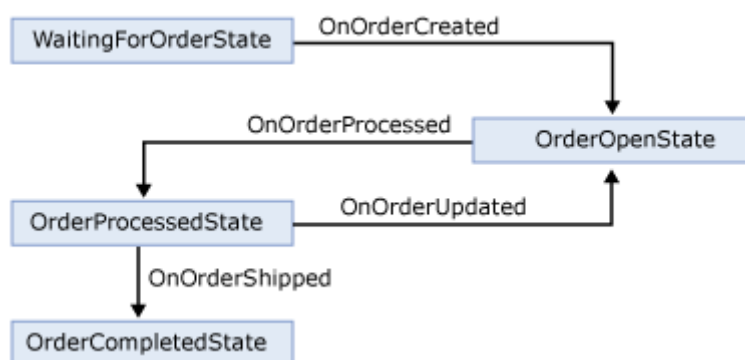
```

Las entidades empresariales se pueden implementar en .NET mediante **ADO.NET Entity Framework**. Permite utilizar clases POCO (*Plain Old C# Objects*), que son clases del modelo de dominio que ignoran la persistencia.

Algunos procesos de negocio están formados por un cierto número de pasos que deben ejecutarse de acuerdo con unas reglas concretas o dependiendo de eventos que se pueden producir en el sistema y, normalmente, con un tiempo de ejecución de larga duración. Este tipo de procesos de negocio se implementan como flujos de trabajo (*workflows*). Hay dos tipos fundamentales de flujo de trabajo: secuencial y máquina de estados. Un **flujo de trabajo secuencial** está diseñado con el fin de ejecutar una serie de pasos predefinidos para llevar a cabo una cierta tarea. Los flujos de trabajo secuenciales constan de una serie de actividades que se ejecutan de manera secuencial hasta que finalice la última actividad. En la siguiente figura se muestra un ejemplo de flujo de trabajo secuencial.



Por el contrario, los **flujos de trabajo de máquina de estados** están orientados a eventos. El flujo de trabajo consta de una serie de estados y de transiciones entre los estados. Todos los estados deben tener al menos una transición, a excepción de un estado final que no puede tener transiciones. Cuando una instancia de flujo de trabajo entra en un estado, se ejecutan las actividades de la acción de entrada. Cuando se completa la acción de entrada, los desencadenadores de las transiciones del estado se programan. Cuando se confirma una transición a otro estado, las actividades de la acción de salida se ejecutan. En la siguiente figura se muestra un ejemplo de este tipo de workflow.



A partir del framework .NET 3.0 se pueden definir workflows mediante **WF** (*Windows Workflow Foundation*). Es necesario hospedar el motor de workflow en una aplicación ASP.NET o Windows. En la instancia del motor de workflow creada, se crea un workflow a partir de una clase y se empiezan a ejecutar sus actividades, como se muestra en el siguiente ejemplo. Las actividades son también clases entre cuyas propiedades se pueden definir dependencias a fin de comunicarse información entre ellas.

```

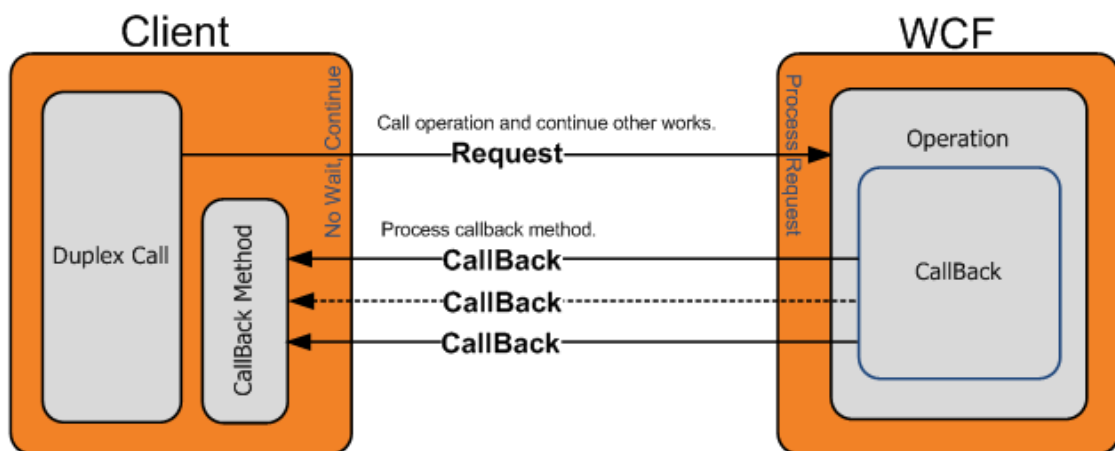
class Program
{
    static void Main(string[] args)
    {
        using(WorkflowRuntime workflowRuntime = new WorkflowRuntime())
        {
            {WorkflowInstance instance =
            workflowRuntime.CreateWorkflow(typeof(MySequentialWorkflow.Workflow1));
            instance.Start();
        }
    }
}

```

Mientras WF permite ejecutar workflows dentro de una aplicación, **Microsoft BizTalk Server** es un servidor de integración que permite automatizar procesos de negocio.

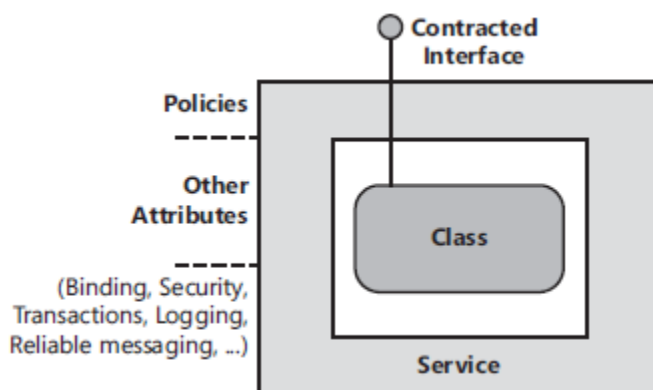
9. La capa de servicios

A la hora de diseñar los servicios hay que tener en cuenta que las funcionalidades que se exponen en los contratos deben ser las realmente demandadas por las aplicaciones clientes - operaciones de grano grueso-. También hay que considerar aspectos como la seguridad en la transferencia de mensajes, la interoperabilidad, los modelos de mensajería, la gestión de excepciones, etc. En los servicios Web XML, tanto el cliente como el servicio intercambian mensajes XML. Existen dos mecanismos de seguridad a este nivel: seguridad a nivel de transporte, mediante **SSL** (*Secure Sockets Layer*); y seguridad a nivel del mensaje, donde se incluyen las credenciales del cliente del servicio y se cifra el mensaje, según la especificación **WS-Security**. Las credenciales pueden ser un usuario/password, un certificado de cliente X.509 o un token de seguridad. WCF permite que cuando una clase proxy llame a un servicio le pase las credenciales de Windows, que el servicio recupera mediante la propiedad *Threading.Thread.CurrentPrincipal*. Respecto a los mecanismos de control de accesos, se puede restringir el acceso a un método del servicio a los clientes no autorizados. También se deben gestionar las excepciones para evitar ataques de denegación de servicio. Respecto a los modelos de intercambio de mensajes, WCF soporta tres modelos: *Petición-Respuesta* o síncrono, donde el cliente queda a la espera de la respuesta del servicio; *asíncrono (one-way)*; y *duplex*, donde tanto el cliente como el servicio pueden efectuar llamadas, que pueden ser síncronas o asíncronas, como se representa en la siguiente figura.



WCF- Duplex (One-Way) Message Exchange Pattern

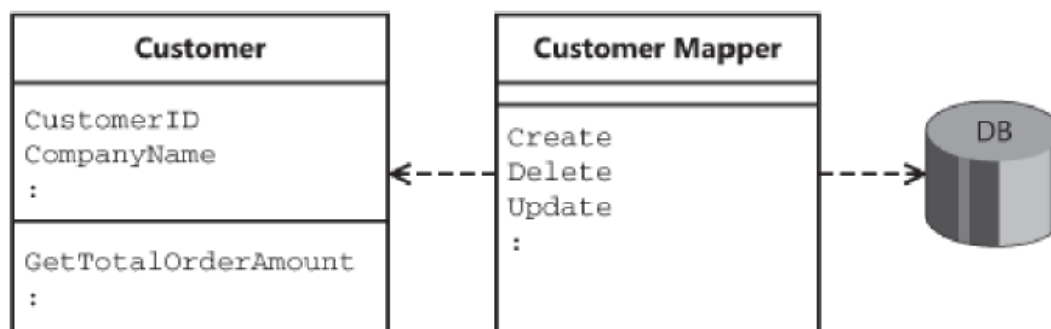
En la siguiente figura se muestra la representación de un servicio como una clase que se ejecuta en un contenedor que tiene capacidades de seguridad, monitorización, búsqueda, mensajería fiable, etc.



Los servicios en .NET se pueden implementar mediante **ASP.NET** o **WCF** (*Windows Communication Foundation*). Estas tecnologías están muy integradas con otras citadas anteriormente, como Silverlight, WF o Microsoft BizTalk Server. Por ejemplo, desde el código AJAX del cliente se puede llamar a un endpoint que sea un servicio Web ASP.NET (.asmx) o WCF (.svc) y refrescar el contenido mostrado en un *TextBlock* de Silverlight.

10. La capa de datos

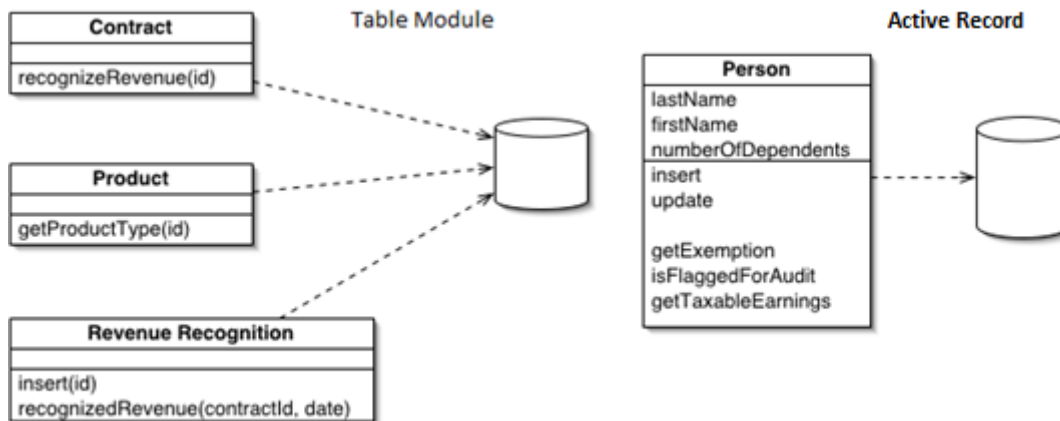
La capa de datos debe ser independiente de la implementación de la base de datos que actúa como repositorio físico de la información. Por independiente nos referimos a la capacidad de la capa de datos de ofrecer los mismos servicios -realizar operaciones CRUD (*Create, Read, Update, Delete*), ejecutar una transacción o una query- sin afectarle cómo se realiza físicamente el almacenamiento de la información. La implementación de los servicios CRUD se realiza normalmente según el patrón de diseño *Data Mapper*, como se muestra en esta figura, donde la lógica de negocio reside en la clase *Customer* la lógica de acceso a datos en la clase *CustomerMapper*.



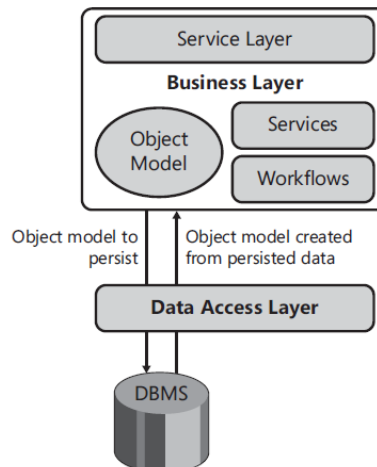
Los servicios CRUD pueden trabajar con objetos persistentes o transitorios. Considerando el ejemplo anterior de la entidad *Customer*, si recuperamos de la base de datos un objeto *Customer*, se trata de un objeto persistente. Si por el contrario creamos un nuevo objeto *Customer* para luego insertarlo en la base de datos, este objeto será transitorio.

Los componentes de acceso a datos pueden estar acoplados en la capa de negocio, como sucede cuando se adoptan los patrones *Table Module* o *Active Record*. En el primero, existe una clase por cada tabla o vista de la base de datos, y una sola instancia de cada clase se encarga de los accesos a datos. En el segundo, por cada fila de una tabla de la base de datos hay una instancia de la clase *Active Record*, por lo que cada objeto se encarga de administrar su propia persistencia (a diferencia de *Data Mapper*, donde un objeto de la clase *PersonMapper* se encargaría de la persistencia).

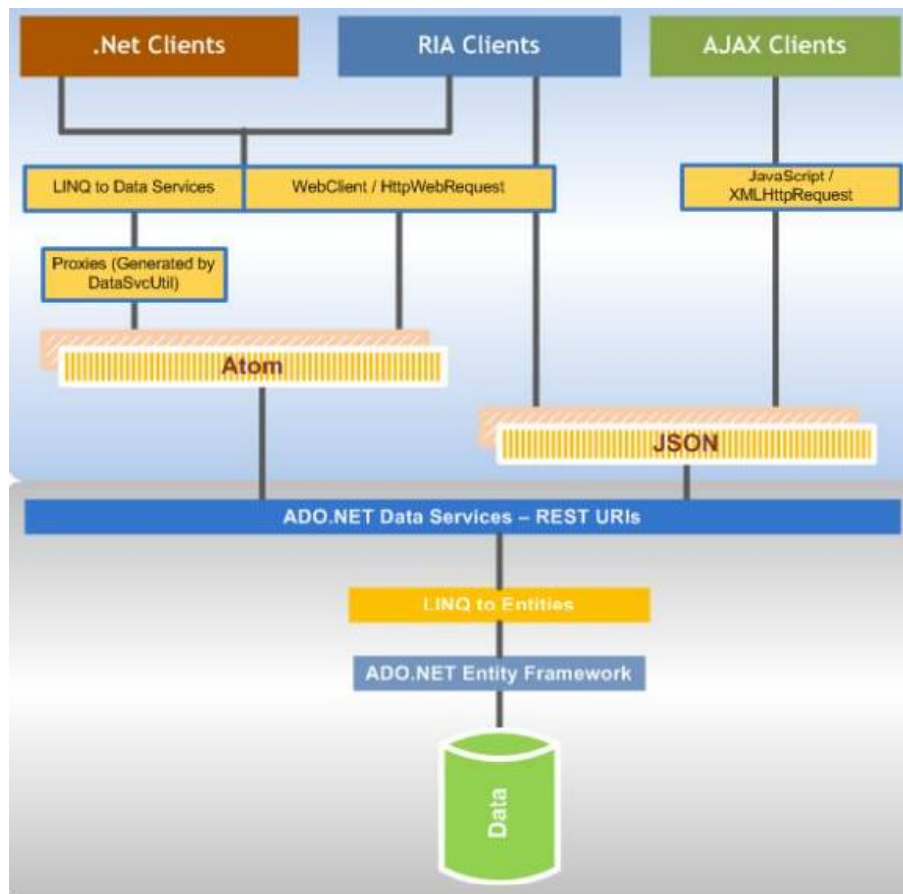
En la siguiente figura se muestran ambos patrones.



Sin embargo, cuando se utiliza el estilo de arquitectura DDD (*Domain Driven Design*), la DAL (*Data Access Layer*) es una herramienta de la capa de datos para la persistencia del modelo de objetos definido en la capa de negocio. Esta idea se plasma en la siguiente figura.



En .NET, la capa de datos se puede implementar con **ADO.NET** y su pila de marcos de trabajo. Especialmente interesante es el caso de las aplicaciones enriquecidas denominadas LOB (*line of business*), que requieren conectarse frecuentemente a los *Data Sources* de la organización. En este caso, se puede crear un modelo de datos **ADO.NET Entity** y exponerlo mediante los **WCF Data Services**, que serán consumidos, por ejemplo, desde Silverlight. Esta arquitectura se expone en la siguiente figura.



11. Arquitectura física

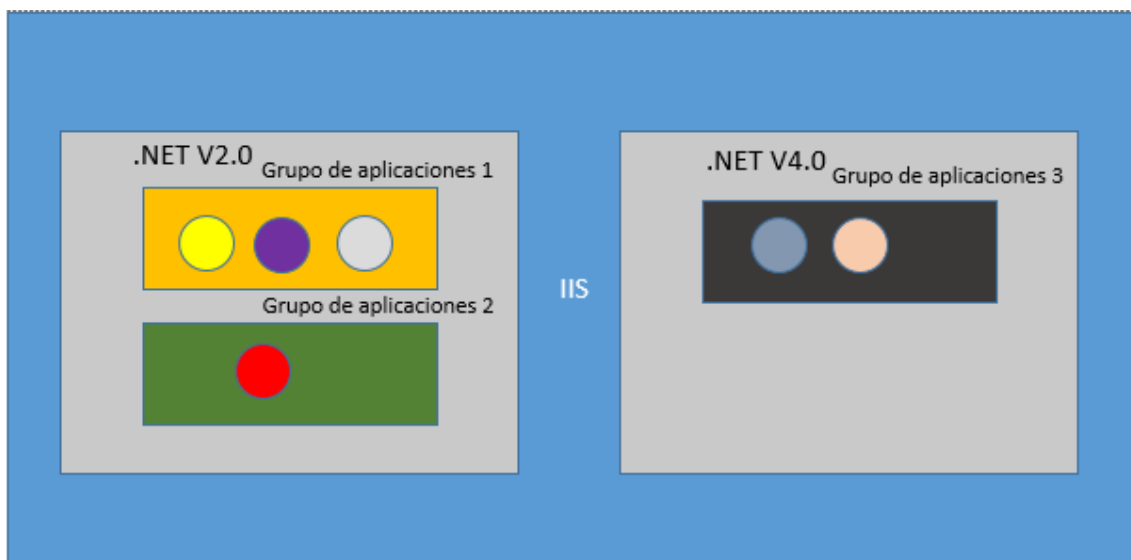
Las capas explicadas en los apartados anteriores finalmente se despliegan en nodos físicos interconectados según las llamadas que efectúan los componentes de las diferentes capas. Entre los diferentes productos software que actúan como contenedores de estos componentes, se pueden destacar en .NET los siguientes: **Windows Server**, **IIS** (*Internet Information Services*), **SQL Server** y otros servidores, como **Microsoft SharePoint** o **Microsoft BizTalk Server**.

Las aplicaciones Web ASP.NET o los servicios Web ASP.NET o WCF se despliegan en IIS. En concreto, se ejecutan en un sitio web (host:puerto) dentro de un grupo de aplicaciones (*application pool*).

Los **grupos de aplicaciones** ofrecen las siguientes ventajas:

- *Mejor rendimiento del servidor y de las aplicaciones:* Se pueden asignar las aplicaciones que usan una gran cantidad de recursos a sus propios grupos de aplicaciones de modo que no disminuye el rendimiento de las otras aplicaciones.
- *Mayor disponibilidad de las aplicaciones:* Si una aplicación de un grupo de aplicaciones genera un error, no se verán afectadas las aplicaciones en otros grupos de aplicaciones.
- *Mayor seguridad:* Al aislar las aplicaciones, se reduce la posibilidad de que una aplicación obtenga acceso a los recursos de otra aplicación.

Cada aplicación ASP.NET se configura de modo que se ejecute con una versión concreta de **CLR** (*Common Language Runtime*). Cuando hay varias versiones de CLR ejecutándose en paralelo en un servidor, cada grupo de aplicaciones se puede asociar a una versión de CLR concreta. Los grupos de aplicaciones se ejecutan en uno de los dos modos de canalización siguientes: el integrado o el clásico. El modo de canalización del grupo de aplicaciones afecta a la forma en que el servidor procesa las solicitudes de código administrado. En la siguiente figura se explica que IIS hospeda varios CLR, dentro de los cuales se pueden ejecutar diferentes grupos de aplicaciones.



Para dotar de **alta disponibilidad** a IIS hay que configurarlo en clúster. En Windows Server 2012 existen dos tipos de clúster: clúster **failover** o clúster de **equilibrio de carga de red**. Se requiere que ambos servidores pertenezcan al mismo dominio. Una solución es crear un dominio y promocionar los servidores que forman el clúster a controladores del mismo. Al clúster se le asignará una IP que será la que haya que utilizar para acceder a las aplicaciones. De requerirse un almacenamiento común entre los nodos del clúster, se puede crear un recurso compartido de CIFS (*Common Internet File System*) o una exportación de NFS (*Network File System*) en un clúster NAS (*Network Attached Storage*).

Para concluir la exposición de este tema, se hará una breve referencia a una aplicación, desarrollada íntegramente en .NET, ampliamente utilizada en la Administración. Se trata de la aplicación **PRO@** (*PROcedimientos Administrativos internos*), que permite la tramitación, a través de Intranet, de los principales procedimientos administrativos internos de un Organismo, mediante el uso de la firma electrónica.

PRO@ es un sistema formado por varias aplicaciones Web y servicios Web. Este sistema requiere su interconexión, a nivel interno, con los servidores LDAP y de correo del organismo, entre otros, y, a nivel externo, con sistemas como Badaral, SOROLLA o @Contrata. Cada aplicación Web tiene un objeto determinado; así, por ejemplo, existe una aplicación Web para la tramitación de solicitudes, como las solicitudes de asistencia a cursos de formación o las solicitudes de vacaciones, que además permite adjuntar ficheros; también hay un portafirmas electrónico, para firmar los documentos asociados a los flujos de trabajo definidos para cada tipo de solicitud. Estas aplicaciones Web están basadas en formularios ASP.NET Web Forms con código Javascript. Constan de páginas .aspx, controles de usuario .ascx y clases compiladas como ensamblados .dll: clases de código asociadas, clases de negocio, clases de acceso a datos (se utiliza una base de datos Oracle) y clases proxy de servicios Web. Por otra parte, también se despliegan varios servicios Web ASP.NET para recibir notificaciones de los sistemas externos y para la consulta de los datos administrados por PRO@. A nivel de la arquitectura física, todas las aplicaciones y servicios Web se ejecutan en un grupo de aplicaciones .NET 3.5, consiguiendo la alta disponibilidad mediante un clúster de equilibrio de carga de red.

12. Bibliografía

- *CLR via C#*. Jeffrey Richter.
- *"ASP.NET 4. Unleashed"*. Stephen Walther, Kevin Hoffman and Nate Dudek.
- *"Application Architecture for .NET: Designing Applications and Services"*. Microsoft.
- *"Microsoft Application Architecture Guide, 2nd Edition"*. Microsoft.
- *"Microsoft .NET: Architecting Applications for the Enterprise"*. Andrea Saltarello and Dino Esposito.
- <http://msdn.microsoft.com>